FUJINET

The FujiNet BASIC Extension

for Applesoft BASIC and ProDOS 8

```
 10 D$ = CHR$(4): Q$ = CHR$(34)
 20 U$ = "N:HTTPS://ICANHAZIP.COM/"
 30 PRINT D$;"NOPEN 1,";Q$;U$;Q$;",4,0"
 40 PRINT D$;"NSTATUS 1,BW,CN,ER"
 50 IF BW = 0 AND CN THEN 40
 60 PRINT D$;"NREAD 1,A$,BW"
 70 PRINT "MY IP IS ";A$
 80 PRINT D$;"NCLOSE 1"

]RUN
MY IP IS 47.190.140.76

; the world, one PRINT CHR$(4); at a time.
```

A command reference for the twenty network commands the FujiNet adds to Applesoft BASIC.

Free Software

FujiNet's firmware, this BASIC extension, and this manual are free software, built and given away by a worldwide community of Apple II owners. You may copy this book for a friend — in fact, we'd be delighted. Source for everything, this booklet included, lives at `github.com/FujiNetWIFI`.

How This Book Was Verified

Every command, argument, error message, and address in this reference was taken from the extension's own source — reprinted in full in Appendix D, copied from the tree each time this book is typeset — and every command was exercised on running hardware against the FujiNet firmware. The extension lives in `fujinet-nhandler/apple2` the firmware side of each operation is in `fujinet-firmware` (`lib/device/iwm/network.cpp` and friends). The assembly-level story of the same machinery is told in the companion volume, *Programming the FujiNet*.

Limitation of Warranties

Neither the FujiNet community nor its contributors make any warranty with respect to this manual or to FujiNet. Everything is provided "as is." But unlike 1984, when something bothers you, you can read the source — it's in the back of this very book — fix it, and send a pull request.

Trademarks

Apple, the Apple logo, Applesoft, ProDOS, and SmartPort are trademarks of Apple Inc. FujiNet is a community project and is not affiliated with, endorsed by, or sponsored by Apple Inc.

Conventions

Program listings and screen transcripts are set in the genuine Apple II character set. Hexadecimal numbers are written with a leading dollar sign, as the Apple II has always written them: `$7C00`, `$BE06` — though as befits a BASIC manual, you will mostly meet honest decimal here. In syntax lines, UPPERCASE is typed as shown and lowercase names stand for the parts you supply.

Dedicated to everyone who ever typed `PRINT CHR$(4)` and trusted that somebody, somewhere, was listening.

FUJINET

The FujiNet BASIC Extension

A command reference for Applesoft BASIC under ProDOS 8: twenty network commands and a printer, for every Apple II with a FujiNet on its SmartPort.

This book is the BASIC-speaking member of a family. *Getting Started with FujiNet* taught your fingers; *Programming the FujiNet* taught your assembler; this one teaches the language the Apple II booted into every morning. By the last chapter you will have fetched a web page, parsed JSON, served TCP, and printed over WiFi — without leaving the] prompt.

Contents

Preface

The Shape of the Thing

Twenty commands, one printer, no new dialect: it is still Applesoft, still ProDOS, still the machine you know.

Somewhere between 1977 and now, the Apple II learned to talk to the whole world. The FujiNet did that: a WiFi peripheral that hangs off the SmartPort and speaks TCP, HTTP, JSON, and half a dozen other protocols on the machine's behalf. What this book documents is the thinnest possible bridge between that power and the language most Apple II programs were ever written in.

The FujiNet BASIC extension adds **twenty network commands** to Applesoft BASIC. They are not new keywords baked into a new interpreter — Applesoft lives in ROM, and its keyword table is carved in silicon. Instead they are **BASIC.SYSTEM external commands**, exactly the mechanism ProDOS provides for the purpose, and they behave like the disk commands you already know: type them at the] prompt, or issue them from a program with `PRINT CHR$(4)`.

Where the Commands Came From

The command set began as a port. The Coleco Adam's SmartBASIC got the core commands first, and this extension deliberately matches that set, name for name — `NOPEN` through `NHTTPMODE` — so programs and habits move between the machines. The Apple II then adds a few of its own: `NCAT` and `NCATALOG`, which print directories in the ProDOS `CAT` and `CATALOG` styles this machine expects, and `NTYPE` and `NTRANS`, for reading a text file straight to the screen. If you have used FujiNet from BASIC anywhere, you already know most of this book.

What You Should Already Know

Applesoft, at the level of *Basic Programming With ProDOS*: variables, strings, `PRINT`, `ONERR`. Nothing here requires assembly language, PEEKs, or POKEs — the two `CALL`s in Chapter 8 are the entire machine-language surface of this book. When you get curious about what is underneath, Appendix C sketches the machinery and Appendix D reprints every line of it.

How the Reference Is Laid Out

Each command gets an entry like this: the name, with a red chip naming the FujiNet operation it rides on (the same operation names used in *Programming the FujiNet*, so the two books cross-reference); a syntax line, where `UPPERCASE` is typed as shown and lowercase

stands for what you supply; a parameter table; the errors it can raise; and a worked example — usually a transcript from a real session, green phosphor and all.

Chapter 1

Getting Started

```
]PR#6

FUJINET BASIC EXTENSION INSTALLED
20 COMMANDS - NOPEN..NTYPE
FUJINET NETWORK DEVICE AT SP UNIT $0B
FUJINET PRINTER AT SP UNIT $0D

]
```

What You Need

- An Apple II that boots ProDOS — any model with 64K.
- A FujiNet on the SmartPort: the real device on a IIc, IIGS or SmartPort card, or `fujinet-pc` behind an emulator.
- The extension disk, `FUJIAPPLE.PO`. It boots ProDOS 8 and BASIC.SYSTEM, then installs the commands automatically.

Booting It

Boot the disk. ProDOS starts BASIC.SYSTEM, BASIC.SYSTEM runs `STARTUP`, and `STARTUP` does one interesting thing:

```
PRINT CHR$(4);"BRUN FUJIAPPLE"
```

That single line is the whole installation. The extension announces itself with the banner shown above: twenty commands are live, and the FujiNet's network device (and printer, if the firmware exposes one) have been found on the SmartPort and remembered.

To install onto your own work disk, copy the `FUJIAPPLE` file across and add the `BRUN` line to your own `STARTUP`. The extension is a single ProDOS `BIN` file; there is nothing else to carry.

No FujiNet? The banner says `FUJINET NETWORK DEVICE NOT FOUND` and every N-command answers `NO DEVICE CONNECTED`. The rest of the machine is unaffected.

What Gets Installed, and Where

The extension is **resident**: it loads once at `$8000` and stays until you reboot. To protect itself it moves BASIC's ceiling, `HIMEM`, down to 31744 (`$7C00`). Your program, variables, and strings live below that line exactly as before; you simply have about six fewer K to spend. A 48K machine still leaves you roughly 29K of program space — the same arithmetic as any resident DOS-era utility, and Appendix C has the full memory map.

Everything else you know keeps working. `CATALOG`, `SAVE`, `LOAD`, `OPEN`, `EXEC` — the ProDOS commands and the N-commands share the same command line, the same error style, and the same disk, without stepping on each other.

Your First Command

Type this at the `]` prompt (substitute any TNFS host you like — `FUJINET.ONLINE` is always awake):

```
]NCAT "N:TNFS://FUJINET.ONLINE/"
/

 NAME            TYPE BLOCKS  MODIFIED

 APOD            DIR       1  17-JUL-26
 CATALOGS        DIR       1  17-JUL-26
 GAMES           DIR       1  17-JUL-26
 README.TXT      TXT       4  17-JUL-26

BLOCKS FREE: 65535  BLOCKS USED: 7

]
```

That listing came over WiFi, from a filesystem on the other side of the internet, into a machine whose designers were worried about the price of 16K RAM chips. It took one line of BASIC.

> **By the Way:** Every command in this book works the same typed at the prompt or issued from a program. The transcripts show the prompt because it is the fastest way to explore; your programs will use `PRINT CHR$(4)`, described next.

Chapter 2

Command Conventions

Typing Commands, Running Commands

At the] prompt, an N-command is typed bare, like CATALOG:

```
NOPEN 1,"N:HTTPS://ICANHAZIP.COM/",4,0
```

Inside a program, an N-command is **printed** — the standard ProDOS idiom, a PRINT whose first character is CTRL-D, CHR$(4):

Listing 2-1. The CHR\$(4) idiom

```
10 D$ = CHR$(4)
20 PRINT D$;"NOPEN 1,";CHR$(34);
   "N:HTTPS://ICANHAZIP.COM/";CHR$(34);",4,0"
```

CHR$(34) is the quotation mark. Building specs into a string variable first (U$) usually reads better — every command that takes a quoted string also accepts a string variable in its place.

Two habits from the disk commands carry over exactly: the CTRL-D must be the first character printed on its line, and the command text is ordinary uppercase.

Channels

The FujiNet holds up to **fifteen simultaneous connections**, numbered 1 through 15. Every connection-oriented command names its channel first: NOPEN 1,… opens channel 1; NREAD 1,… reads it; channels 2 through 15 are yours for more connections at the same time — a web fetch on one, a TCP chat on another. A channel number outside 1–15 raises RANGE ERROR.

> **By the Way:** The extension reserves channel 0 for its own housekeeping — the filesystem commands of Chapter 4 ride on it — which is why you cannot open it yourself.

The Device Spec

Every place this book says *spec*, it means a FujiNet device specification: the string that names what to connect to, always beginning N:.

```
N:protocol://host[:port]/path
```

The firmware speaks a long list of protocols and the extension does not care which — it just hands the spec across. The ones that present a **filesystem**, with directories you can list, walk, and read, work with every command in Chapter 4:

Protocol	Talks to	Example
TNFS	TNFS file servers	N:TNFS://FUJINET.ONLINE/GAMES/
HTTP	web servers	N:HTTP://IP-API.COM/JSON/
HTTPS	web servers, TLS	N:HTTPS://ICANHAZIP.COM/
FTP	FTP servers	N:FTP://FTP.GNU.ORG/
SFTP	SSH file transfer	N:SFTP://USER:PASS@HOST/PATH
SMB	Windows / Samba shares	N:SMB://SERVER/SHARE/
NFS	NFS exports	N:NFS://SERVER/EXPORT/
S3	Amazon S3 buckets	N:S3://ENDPOINT/BUCKET/
SD	the FujiNet's SD card	N:SD:///GAMES/
GDRIVE	Google Drive	N:GDRIVE:///PATH/
ONEDRIVE	Microsoft OneDrive	N:ONEDRIVE:///PATH/
GMAIL	read Gmail folders	N:GMAIL:///INBOX
IMAPS	read IMAP mail, TLS	N:IMAPS://USER:PASS@HOST/INBOX

The rest are **streams** — a byte pipe with no directory. They open, carry data, and close; the filesystem commands do not apply:

Protocol	Talks to	Example
TCP	raw sockets, or listen	N:TCP://BBS.EXAMPLE.ORG:6502/
UDP	datagrams	N:UDP://192.168.1.7:5000/
TELNET	telnet hosts	N:TELNET://RETRO.SDF.ORG:23/
SSH	SSH sessions	N:SSH://USER@HOST:22/
WS / WSS	WebSocket servers	N:WS://ECHO.EXAMPLE.ORG/
CPM	the built-in CP/M	N:CPM://

> **By the Way:** Where a protocol needs a login it rides in the spec: `USER:PASS@HOST` gives a password, and `SFTP` and `SSH` also accept a bare `USER@HOST`, falling back to the private key the FujiNet keeps at `/.ssh/id_ed25519` on its SD card. `IMAPS` and `S3` likewise read credentials from the userinfo (or the FujiNet's own configuration) when you leave them out.

> **By the Way:** `GMAIL`, `GDRIVE`, and `ONEDRIVE` carry no password at all — you authorize the FujiNet once in its own web configuration, then address them with the empty-host, triple-slash form above. `GMAIL` is read-only: a folder name is a Gmail label, and the path chooses depth — `N:GMAIL:///INBOX` counts the folder, `N:GMAIL:///INBOX/1` reads the newest message, and `N:GMAIL:///INBOX/1/2` an attachment.

A spec appears in a command either as a quoted literal or as a **simple string variable** — an expression such as `A$+B$` will not do:

```
NOPEN 2,U$,4,0
```

Modes and Translation

NOPEN takes two small numbers after the spec. The **mode** says which way data flows; **translation** rewrites line endings between the Apple's carriage returns and the network's conventions.

Mode	Meaning
4	read
8	write
12	read/write
13	HTTP POST

Trans	Line endings
0	none (binary)
1	CR (Apple)
2	LF (Unix)
3	CR/LF (internet)
4	PETSCII (Commodore)

For text from the modern world, mode 4 with translation 2 or 3 is the usual choice; for binary data, always translation 0.

When Something Goes Wrong

The N-commands report trouble the ProDOS way — a message at the prompt, or a catchable error under ONERR GOTO:

Message	PEEK(222)	Meaning
NO DEVICE CONNECTED	3	no FujiNet found at boot
RANGE ERROR	2	channel number not 1–15
I/O ERROR	8	the FujiNet refused the operation
PROGRAM TOO LARGE	14	NLOAD: file exceeds program space

A mistyped argument — a missing comma, a word where a number should be, an argument too many — is reported as a plain ?SYNTAX ERROR (code 16 under ONERR); the command does not run, and nothing else is disturbed.

Important!

> Network trouble on an open channel — a dropped connection, a server error — does *not* interrupt your program. It shows up in the error variable of NSTATUS, which well-behaved programs check as they go. Chapter 3 shows the pattern.

Chapter 3

Connections

Five commands carry every conversation: open a channel, ask it how it is doing, read it, write it, close it. Learn these five and the rest of the book is detail.

NOPEN

CONTROL 'O'

```
NOPEN channel, spec, mode, trans
```

Opens *channel* to the destination named by *spec*. The connection is attempted immediately.

Argument	Type	Meaning
channel	number 1-15	which of the fifteen channels to open
spec	string	the device spec (Chapter 2)
mode	number	4 read, 8 write, 12 read/write, 13 POST
trans	number	0 none, 1 CR, 2 LF, 3 CR/LF

Errors RANGE ERROR for a bad channel. For file-flavored protocols — TNFS and its kin — a failed open is reported at once: PATH NOT FOUND if the file or directory is not there, I/O ERROR for other refusals, and the channel is closed for you. For HTTP and HTTPS the request has not actually fired yet (see Chapter 7), so nothing can have failed: check the error value of NSTATUS after your first read.

NCLOSE

CONTROL 'C'

```
NCLOSE channel
```

Closes the channel and frees its connection on the FujiNet. Close what you open — fifteen channels feels infinite right up until it isn't.

NSTATUS

STATUS 'S'

```
NSTATUS channel, bw, conn, err
```

The heartbeat of every network program. Asks the FujiNet how *channel* is doing and stores the answer into three *numeric variables you name* — they are assigned, not read:

Variable	Receives	Meaning
bw	0-65535	bytes waiting to be read
conn	0 or 1	1 while the connection is alive
err	code	1 = all is well; 136 = end of data; 128+ = trouble

The idiomatic fetch loop asks three questions in order: anything to read? still connected? did it end well?

Listing 3-1. The NSTATUS loop, the shape of every fetch

```
100 PRINT D$;"NSTATUS 1,BW,CN,ER"
110 IF BW > 0 THEN GOSUB 200: GOTO 100
120 IF CN = 1 THEN 100
130 IF ER <> 136 THEN PRINT "ERROR ";ER
140 END
```

NREAD

READ

```
NREAD channel, var$, count
```

Reads up to *count* bytes (1 to 255) from the channel into the string variable *var$*. The variable receives *exactly what arrived* — if fewer bytes were waiting, it is shorter; check `LEN(var$)`, not your request. Ask `NSTATUS` first and read the smaller of *bw* and 255.

NWRITE

WRITE

```
NWRITE channel, data$, count
```

Writes the first *count* bytes of *data$* to the channel. If *count* exceeds `LEN(data$)`, the whole string is written and nothing more. For strings, the natural spelling is `NWRITE 1,A$,LEN(A$)`… except that *count*, like every numeric argument, may be any expression — so that spelling works as written.

A Complete Exchange

The program from the front cover, in full. It opens an HTTPS channel, waits for the reply, reads it, and says where you are:

Listing 3-2. WHATSMYIP — a complete HTTPS fetch

```
10 D$ = CHR$(4): Q$ = CHR$(34)
20 U$ = "N:HTTPS://ICANHAZIP.COM/"
30 PRINT D$;"NOPEN 1,";Q$;U$;Q$;",4,0"
40 PRINT D$;"NSTATUS 1,BW,CN,ER"
50 IF BW = 0 AND CN = 1 THEN 40
60 IF BW = 0 THEN PRINT "NO REPLY, ERR ";ER: GOTO 90
70 PRINT D$;"NREAD 1,A$,BW"
```

```
80 PRINT "MY IP IS ";A$
90 PRINT D$;"NCLOSE 1"
```

```
]RUN
MY IP IS 47.190.140.76

]
```

> **By the Way:** Line 50 is the polite way to wait: while the reply is still in flight, *bw* is zero but *conn* is one. When both go to zero without data, something failed — and line 60 reports the code instead of looping forever.

Chapter 4

The Filesystem Commands

Six commands treat the network as a disk: list it two ways, walk it, make and remove directories, delete files. They take **no channel number** — just a spec — because each is a complete errand: the extension runs it on a reserved internal channel and is done before the prompt returns.

They work on any filesystem-shaped protocol — TNFS above all, and anything else the firmware exposes with directories.

NCAT

OPEN 6 · CAT

```
NCAT spec
```

Prints the directory named by the spec to the screen in the ProDOS `CAT` layout: a 40-column table of name, type, size in blocks, and date, that fits the plain text screen. This is the command from Chapter 1 — the fastest way to see whether the network is alive.

NCATALOG

OPEN 6 · CATALOG

```
NCATALOG spec
```

The same listing in the wider ProDOS `CATALOG` layout, which adds created-date, end-of-file, and subtype columns and wants an 80-column display. Both commands run on the reserved internal channel in directory mode; only the format they ask the FujiNet to send back differs.

NCD

CONTROL ','

```
NCD spec
```

Sets the working prefix for the filesystem commands that follow, the way `PREFIX` does for ProDOS. Two forms are worth knowing:

```
NCD "N:TNFS://FUJINET.ONLINE/GAMES/"
NCD "N:.."
```

The second walks up one level, wherever you are.

NMKDIR

CONTROL '*'

```
NMKDIR spec
```

Creates the directory named by the spec.

NRMDIR

CONTROL '+'

```
NRMDIR spec
```

Removes the (empty) directory named by the spec.

NDEL

CONTROL '!'

```
NDEL spec
```

Deletes the file named by the spec.

Errors All six raise catchable errors when the server refuses. `NCAT` and `NCATALOG` answer `PATH NOT FOUND` for a directory that isn't there; the others answer `I/O ERROR` — a missing path, a directory that isn't empty, a share without write permission.

A Working Session

```
]NMKDIR "N:TNFS://TMA-3/SCRATCH"

]NCAT "N:TNFS://TMA-3/"
/

 NAME            TYPE BLOCKS  MODIFIED

 BIN             DIR       1  17-JUL-26
 SCRATCH         DIR       1  17-JUL-26
 SRC             DIR       1  17-JUL-26
 HELLO.BAS       BAS       2  17-JUL-26

BLOCKS FREE: 65535  BLOCKS USED: 5

]NRMDIR "N:TNFS://TMA-3/SCRATCH"

]
```

Viewing a Text File

Two more commands read a text file straight to the screen, so you can glance at a README, a web page, or a piece of mail without writing a `NREAD` loop.

NTYPE

OPEN 4 + READ

```
NTYPE spec
```

Opens the spec, prints its bytes to the screen as they arrive, and closes it. `NTYPE` does nothing to the text beyond the translation you set with `NTRANS` — it simply reads and shows. While it runs, `CTRL-S` pauses the scroll and `CTRL-C` stops it and returns to the prompt. It is content with anything textual: a `TNFS` README, an `HTTPS` page, JSON, or a `GMAIL` message body.

`NTYPE` borrows channel 1 for the read. If you are keeping a connection open there, close it first.

Errors `PATH NOT FOUND` when the file is not there; `I/O ERROR` when the server refuses.

NTRANS

LOCAL

```
NTRANS ch, mode
```

Records the line-ending translation `NTYPE` will use on its next open. The *mode* is one of the translation numbers you already know from `NOPEN`, plus one more for Commodore text:

mode	Line endings
0	none (binary)
1	CR (Apple)
2	LF (Unix)
3	CR/LF (internet)
4	PETSCII (Commodore)

> **By the Way:** `NTRANS` sends nothing over the wire. The FujiNet can only choose a translation when a channel is opened, so the command just remembers the number for `NTYPE` to hand over on its next open. The channel argument is accepted for symmetry with the other commands, but a single setting serves `NTYPE`, and it begins life at `0` — no translation.

Set the translation once, then type a file:

```
]NTRANS 1,2

]NTYPE "N:TNFS://FUJINET.ONLINE/README.TXT"
FUJINET.ONLINE PUBLIC TNFS SERVER

BE KIND. KEEP UPLOADS UNDER 10 MB.
MORE AT FUJINET.ONLINE.

]
```

Chapter 5

Programs over the Network

Two commands make the network a place to keep BASIC programs. Your program library can live on a TNFS server — one copy, reachable from every Apple II in the house, or the hemisphere.

NSAVE

OPEN 8 + WRITE

```
NSAVE spec
```

Writes the program in memory to the network file named by the spec, in Applesoft's own tokenized form — the same bytes `SAVE` puts on disk. The file it makes is what `NLOAD` takes back.

```
NSAVE "N:TNFS://TMA-3/LIB/STARBASE.BAS"
```

NLOAD

OPEN 4 + READ

```
NLOAD spec
```

Replaces the program in memory with the network file named by the spec, relinks it, and clears variables — afterwards, `LIST` and `RUN` behave exactly as after a disk `LOAD`.

Important!

> `NLOAD` is for the `]` prompt only. Like ProDOS's own rule for a program that replaces itself, issuing it from a running program would pull the program out from under its own feet.

Errors `PATH NOT FOUND` if the file is not there — and the program in memory is left untouched. `I/O ERROR` for other refusals. `PROGRAM TOO LARGE` if the file will not fit below `HIMEM` — the partial load is discarded and memory is left clean, as after `NEW`.

```
]10 PRINT "HELLO FROM THE NET"
]NSAVE "N:TNFS://TMA-3/HELLO.BAS"

]NEW

]NLOAD "N:TNFS://TMA-3/HELLO.BAS"

]RUN
HELLO FROM THE NET

]
```

> **By the Way:** The saved image is a tokenized Applesoft program, not text — compact, exact, and instantly loadable, but meant for Apple IIs. To publish source for other machines to read, print the listing through the FujiNet printer (Chapter 8) instead.

Chapter 6

Reading JSON

The modern web answers in JSON — nested braces and quoted keys, easy for a server, miserable for a 6502. So the 6502 doesn't do it. The FujiNet parses the document **on the device**, and BASIC asks for values by name, one at a time, each arriving as a tidy string.

The dance is always: `NOPEN` the document, `NJSONPARSE` it, then `NJSONQUERY` for each value you want.

NJSONPARSE

CONTROL $FC + 'P'

```
NJSONPARSE channel
```

Switches an open channel into JSON mode and parses the document it is reading. The channel must already be `NOPEN`ed (mode 4) on something that serves JSON. Parse once; query as often as you like.

NJSONQUERY

CONTROL 'Q' + READ

```
NJSONQUERY channel, result$, query
```

Looks up one value in the parsed document and assigns it, as text, to the string variable *result$*. The *query* is a slash path from the top of the document — a quoted literal or a string variable:

Query	Finds
`/city`	the value of key "city" at the top level
`/results/0/name`	key "name" of the first element of array "results"
`/main/temp`	key "temp" inside object "main"

Numbers index into arrays, counting from zero. A value that does not exist comes back as the empty string — test with `LEN(R$)`. Values longer than 255 characters are truncated to fit a BASIC string.

A Worked Example: Where Am I?

`IP-API.COM` answers with your address and whereabouts, in JSON, for free:

Listing 6-1. WHEREAMI — parse and query a live JSON service

```
 10 D$ = CHR$(4): Q$ = CHR$(34)
 20 U$ = "N:HTTP://IP-API.COM/JSON/"
 30 PRINT D$;"NOPEN 1,";Q$;U$;Q$;",4,0"
 40 PRINT D$;"NJSONPARSE 1"
 50 PRINT D$;"NJSONQUERY 1,IP$,";Q$;"/query";Q$
 60 PRINT D$;"NJSONQUERY 1,CI$,";Q$;"/city";Q$
 70 PRINT D$;"NJSONQUERY 1,CO$,";Q$;"/country";Q$
 80 PRINT D$;"NCLOSE 1"
 90 PRINT "YOU ARE ";IP$
100 PRINT "IN ";CI$;", ";CO$
```

```
]RUN
YOU ARE 47.190.140.76
IN DALLAS, UNITED STATES

]
```

> **By the Way:** Ten lines, no string chopping, no bracket counting. Any JSON service on the internet — weather, time, news, your own — is now a BASIC subroutine away.

Chapter 7

Servers, Credentials, and HTTP

Three commands round out the set: one turns your Apple II into a server, one presents credentials, and one gets at the parts of HTTP that live outside the page body.

NACCEPT

CONTROL 'A'

```
NACCEPT channel
```

Accepts an incoming caller on a **listening** TCP channel. To listen, open a TCP spec with no host — just a port — and wait for `NSTATUS` to report a connection:

```
NOPEN 1,"N:TCP://:6502/",12,0
```

When *conn* goes to 1, someone is on the line; `NACCEPT` takes the call, and from then on `NREAD` and `NWRITE` on that channel talk to your caller.

Listing 7-1. ECHO — a TCP server in a dozen lines

```
 10 D$ = CHR$(4)
 20 PRINT D$;"NOPEN 1,";CHR$(34);
    "N:TCP://:6502/";CHR$(34);",12,0"
 30 PRINT "LISTENING ON 6502..."
 40 PRINT D$;"NSTATUS 1,BW,CN,ER"
 50 IF CN = 0 THEN 40
 60 PRINT D$;"NACCEPT 1"
 70 PRINT "CALLER!"
 80 PRINT D$;"NSTATUS 1,BW,CN,ER"
 90 IF BW = 0 AND CN = 1 THEN 80
100 IF BW = 0 THEN PRINT D$;"NCLOSE 1": END
110 PRINT D$;"NREAD 1,A$,BW"
120 PRINT D$;"NWRITE 1,A$,LEN(A$)"
130 PRINT A$;: GOTO 80
```

Aim a telnet or `nc` session at your Apple's address, port 6502, and watch a 1977 machine serve the modern internet, politely repeating everything you say.

NLOGIN

CONTROL $FD $FE

```
NLOGIN channel, user$, pass$
```

Presents a username and password for *channel*'s *next* `NOPEN` — so log in first, then open. Protected TNFS shares and HTTP Basic Authentication both take their credentials this way.

Listing 7-2. Opening a protected resource

```
10 D$ = CHR$(4): Q$ = CHR$(34)
20 PRINT D$;"NLOGIN 1,";Q$;"OPERATOR";Q$;
   ",";Q$;"SECRET";Q$
30 PRINT D$;"NOPEN 1,";Q$;
   "N:HTTP://MY.SERVER/PRIVATE/";Q$;",4,0"
```

NHTTPMODE

CONTROL 'M'

```
NHTTPMODE channel, mode
```

Shifts what an open HTTP channel reads and writes — the page body is mode 0, the machinery around it is the rest:

Mode	The channel now
0	reads the response body (the normal state)
1	collects header names you NWRITE, for the next request
2	reads response headers, one per NREAD
3	sets headers: each NWRITE adds one "Name: value" line
4	takes POST data: NWRITE the form body, then read the reply

The common recipe — custom headers on a request — is: open, mode 3, `NWRITE` each header line, mode 0, then read the body as usual.

> **By the Way:** This recipe works because an HTTP open doesn't fire the request; the FujiNet waits until the first status or read, giving you this window to set headers or POST data. It is also why `NOPEN` cannot report an HTTP failure immediately (Chapter 3) — at open time, nothing has happened yet.

Chapter 8

The Printer

The FujiNet is also a printer — a SmartPort printer device that renders whatever you send into a PDF (or a period-correct emulation of a classic printer) waiting on the FujiNet's web page. The extension connects Applesoft to it with two `CALL`s:

```
CALL 32771      printer ON  - output goes to the FujiNet
CALL 32774      printer OFF - output returns to the screen
```

Between the two calls, everything Applesoft prints — `PRINT` statements, `LIST`, `CATALOG` — goes to the printer instead of the screen, exactly as if you had issued `PR#1` to a printer card in slot 1.

Listing 8-1. A report, on paper(ish)

```
10 CALL 32771
20 PRINT "STARBASE INVENTORY, STARDATE 2026.7"
30 FOR I = 1 TO 5
40 PRINT I;TAB(8);N$(I);TAB(24);Q(I)
50 NEXT I
60 CALL 32774
```

How the Paper Works

Output is gathered a line at a time and sent to the FujiNet at each carriage return (or when a line passes 80 columns). Turning the printer **off** delivers any unfinished line, so a `PRINT` ending in a semicolon is never lost — but the tidiest habit is to let your last `PRINT` end normally before `CALL 32774`.

To print a program listing, use the pair straight from the prompt:

```
]CALL 32771
]LIST
]CALL 32774
]
```

> **By the Way:** While the printer is on, the screen is silent — including the `]` prompts and your own keystrokes, which land on the paper instead. That's normal `PR#`-style behavior, and it's why the transcript above shows only what you type.

Important!

If no printer device was present at boot (the banner had no `FUJINET PRINTER` line), `CALL 32771` does nothing at all — safe, but silent. Check the FujiNet's printer settings and reboot.

Appendix A

Command Quick Reference

Arguments: *ch* a channel number 1–15; *spec* a device spec, quoted or a string variable; *v* a numeric variable that receives a value; *v$* a string variable that receives a value; *n* a number.

Command	Arguments	Does
NOPEN	ch, spec, mode, trans	open a channel to spec
NCLOSE	ch	close the channel
NSTATUS	ch, v, v, v	bytes waiting, connected, error
NREAD	ch, v$, n	read up to n bytes (max 255)
NWRITE	ch, data$, n	write first n bytes of data$
NJSONPARSE	ch	parse the channel's JSON document
NJSONQUERY	ch, v$, query	fetch one JSON value by /path
NCD	spec	set the filesystem working prefix
NCAT	spec	print a 40-column CAT listing
NCATALOG	spec	print an 80-column CATALOG listing
NMKDIR	spec	create a directory
NRMDIR	spec	remove an empty directory
NDEL	spec	delete a file
NTYPE	spec	print a text file to the screen
NTRANS	ch, n	set the translation NTYPE uses
NLOAD	spec	load a program (prompt only)
NSAVE	spec	save the program in memory
NACCEPT	ch	accept a caller on a listening channel
NLOGIN	ch, user$, pass$	credentials for the next NOPEN
NHTTPMODE	ch, n	HTTP body/header/POST sub-mode

Statement	Does
CALL 32771	printer ON: output to the FujiNet printer
CALL 32774	printer OFF: output back to the screen

NOPEN Modes and Translation

Mode	Meaning
4	read
8	write
12	read/write
13	HTTP POST

Trans	Line endings
0	none (binary)
1	CR (Apple)
2	LF (Unix)
3	CR/LF (internet)
4	PETSCII (Commodore)

Appendix B

Errors

Command Errors

These stop the command and print a message — or land in your `ONERR GOTO` handler, where `PEEK(222)` tells them apart:

Message	PEEK(222)	Raised when
NO DEVICE CONNECTED	3	no FujiNet network device was found at boot
RANGE ERROR	2	a channel number is not 1-15
PATH NOT FOUND	6	an open names a file or directory that isn't there
I/O ERROR	8	the FujiNet refused the operation
PROGRAM TOO LARGE	14	an NLOAD file will not fit below HIMEM
?SYNTAX ERROR	16	a command's arguments are malformed

Channel Status Codes

The third variable of `NSTATUS` reports the health of an open channel without interrupting the program:

err	Meaning
1	all is well
136	end of data - the far side finished cleanly
144	a general, fatal error
170	file not found
128-255	other trouble: connection refused, reset...

A robust fetch loop treats 136 as success-and-stop, anything else at or above 128 as failure, and keeps its own counsel about the rest.

Appendix C

The Memory Map, and How It Hooks In

Where Everything Lives

Range	Contents
$0801 up	your Applesoft program, then its variables
down to $7C00	your strings, growing downward
$7C00	HIMEM - BASIC's ceiling while the extension is resident
$7C00-$7FFF	BASIC.SYSTEM's first file buffer lands here
$8000-$95FF	the extension: code, tables, and buffers
$9A00-$BEFF	BASIC.SYSTEM itself
$BF00-$BFFF	ProDOS global page

The cost of the network, then, is about six K of program space next to a bare ProDOS boot — a 48K machine keeps roughly 29K for BASIC.

External Commands, Not Ampersands

Applesoft cannot take new keywords — its tokenizer and keyword table are in ROM. The classic workaround, the ampersand hook, sees the line *after* tokenizing, by which time `NREAD` has a `READ` token buried in it and `NSTATUS` hides an `AT`. Matching mangled bytes is a bug farm, and the first generation of this extension farmed a few.

BASIC.SYSTEM offers the honest path: an **external command** hook (`EXTRNCMD`, `$BE06`) that hands over every command line *before* Applesoft touches it, raw and untokenized, exactly as it does for `CATALOG`. The N-commands are therefore first-class ProDOS commands: same prompt, same `CHR$(4)` idiom, same error discipline.

Three Rules of Residency

For the assembly-minded, the extension survives alongside BASIC.SYSTEM by honoring three contracts, each learned the hard way and documented in the source in the back of this book:

- **Decline with carry set.** BASIC.SYSTEM pre-sets carry and jumps into the external-command hook; carry clear on return means *claimed*. A recognizer that falls off its keyword table with carry accidentally clear silently claims every line typed at the prompt.

- **Keep BASIC.SYSTEM's memory bookkeeping in sync.** Lowering `MEMSIZ` is not enough: BI caches its own HIMEM page (`RSHIMEM`, `\$BEFB`) and restores it when the last file closes. Set both, or the first `CATALOG` un-protects you.
- **Leave the first kilobyte above HIMEM empty.** BI places the first open file's 1K buffer *above* HIMEM — that is what the stock `\$9600-\$99FF` gap is for. A resident parked directly at HIMEM gets a directory block written over its code.
- **Never let a ROM error escape your handler.** The Applesoft evaluators bail out through `ERROR` on bad input, which abandons BI's dispatch mid-command and wedges its command processing until reboot. The extension runs every handler under a private `ONERR` frame aimed at a one-line decoy program whose `CALL` unwinds the stack and fails the command properly — so a typo costs you a `?SYNTAX  ERROR`, not the machine.

The deeper story — SmartPort calls, the NETWORK device, channels and control codes — is the subject of the companion volume, *Programming the FujiNet*.

Appendix D

The Complete Source Listing

In the tradition of the *Monitor ROM listings* Apple shipped with the IIe, here is the extension, whole: every routine, every hard-won comment. Three files of ca65 assembly, built with the cc65 toolchain (`make apple2` in `fujinet-nhandler/apple2` produces the bootable disk). These pages are regenerated from the source tree every time the book is typeset, so what you read here is what you booted.

equ.inc — equates and constants

```
;*****************************************************************************
; equ.inc - Equates for the FujiNet BASIC.SYSTEM extension (Apple II / ProDOS)
;
; Fresh rewrite of the FujiNet Applesoft extension.  Unlike the original
; ampersand version, this installs as a BASIC.SYSTEM *external command*
; (hooking EXTRNCMD at $BE06), so command names are matched against the raw,
; untokenized input line at $200 - the ROM tokenizer never mangles them.
;*****************************************************************************

;----------------------------------------------------------------------------
; Apple II text / character constants
;----------------------------------------------------------------------------
SPACE           = $A0           ; space, high bit set (for COUT)
CR              = $0D           ; carriage return (ASCII)
TOK_CALL        = $8C           ; Applesoft CALL token (for the error-trap line)

;----------------------------------------------------------------------------
; Zero page - "safe" scratch locations (same set the original used)
;----------------------------------------------------------------------------
ZP1             = $EB
ZP1_LO          = $EB
ZP1_HI          = $EC
ZP2             = $ED
ZP2_LO          = $ED
ZP2_HI          = $EE
ZP3             = $FA
ZP3_LO          = $FA
ZP3_HI          = $FB
ZP4             = $FC
ZP4_LO          = $FC
ZP4_HI          = $FD

;----------------------------------------------------------------------------
; Applesoft zero page
;----------------------------------------------------------------------------
TXTTAB          = $67           ; start of the Applesoft program ($0801)
VARTAB          = $69           ; end of program / start of simple variables
ARYTAB          = $6B           ; start of arrays
STREND          = $6D           ; end of arrays / start of free space
FRETOP          = $6F           ; top of string pool (lo/hi)
MEMSIZ          = $73           ; HIMEM (lo/hi) - highest RAM Applesoft may use
CURLIN          = $75           ; current line # ($FFxx = direct/immediate mode)
VARPTR          = $83           ; PTRGET result: address of the located variable
ERRFLG          = $D8           ; bit 7 set = ONERR armed
TRCFLG          = $F2           ; bit 7 set = TRACE on.  NOTE: BASIC.SYSTEM
                                ; parks a sentinel ($A5 - bit 7 set!) here while
                                ; processing a command, so Applesoft run inside
                                ; that window traces unless we silence it
ONRTXT          = $F4           ; ONERR handler text ptr; HANDLERR -> TXTPTR
```

```
ONRLIN          = $F6           ; ONERR handler line #;  HANDLERR -> CURLIN
DSCTMP          = $9D           ; string descriptor scratch (shares FAC space)
CHRGET          = $B1           ; advance TXTPTR, return char in A
CHRGOT          = $B7           ; re-fetch current char (no advance)
TXTPTR          = $B8           ; Applesoft "current text" pointer (lo/hi)

;-------------------------------------------------------------------------
; Applesoft ROM entry points
;-------------------------------------------------------------------------
; The error flow (verified from the Apple IIe ROM):
;   SYNERR $DEC9:   LDX #$10 / JMP ERROR
;   ERROR  $D412:   BIT ERRFLG / BPL print-and-RESTART / JMP HANDLERR
;   HANDLERR $F2E9: $DE <- X (code); TXTPTR <- (ONRTXT); CURLIN <- (ONRLIN);
;                   JSR CHRGOT; JSR GOTO; JMP NEWSTT   (no stack unwind)
; CMDENTRY exploits this to trap ROM errors raised inside a command handler.
GETBYT          = $E6F8         ; evaluate expression -> byte in X
CHKCOM          = $DEBE         ; require a comma at TXTPTR
PTRGET          = $DFE3         ; locate/create named variable; addr -> VARPTR
STRINI          = $E3D5         ; make room for a new string
MOVSTR          = $E5E2         ; copy string into string space
GIVAYF          = $E2F2         ; signed word (A=hi,Y=lo) -> FAC
MOVMF           = $EB2B         ; pack FAC -> memory at (Y,A) [X too]

;-------------------------------------------------------------------------
; Monitor ROM
;-------------------------------------------------------------------------
CSW             = $36           ; character output vector (COUT jumps here);
                                ; $36-$39 = CSW+KSW, owned by BI's wedge
COUT            = $FDED         ; print A to current output
CROUT           = $FD8E         ; print carriage return
PRTX            = $F944         ; print X as 2 hex digits
KBD             = $C000         ; keyboard data; bit7 set = key ready, low 7 = ASCII
KBDSTRB         = $C010         ; keyboard strobe; access clears the key-ready flag
CTRL_C          = $83           ; CTRL-C, high bit set (as read from KBD)
CTRL_S          = $93           ; CTRL-S, high bit set (as read from KBD)

;-------------------------------------------------------------------------
; ProDOS / BASIC.SYSTEM
;-------------------------------------------------------------------------
IN              = $0200         ; BASIC.SYSTEM/monitor input buffer (hi bit set)

; --- BASIC.SYSTEM global page ($BE00-$BEFF) ---
EXTRNCMD        = $BE06         ; external-command hook.  BI does SEC then
                                ; tail-JMPs here after its internal command
                                ; table fails: carry CLEAR on return = claimed,
                                ; carry SET = declined (MUST be explicit!)
OUTVECT1        = $BE12         ; BI per-slot output vector table ($BE10+2n).
                                ; NOTE: planting a RAM routine here does NOT
                                ; make PR#1 work - BI's PR# validates the vector
                                ; and rejects non-slot-ROM addresses with NO
                                ; DEVICE CONNECTED (ProDOS 2.4.3).  Unused; kept
                                ; as hard-won documentation.
VECOUT          = $BE30         ; BI's ACTIVE output vector (default COUT1 $FDF0);
                                ; BI sends every char here after wedge processing.
                                ; THE working hook point for output redirection.
XTRNADDR        = $BE50         ; post-parse jump target (lo/hi)
XLEN            = $BE52         ; command keyword length - 1
XCNUM           = $BE53         ; command number (0 for external)
PBITS           = $BE54         ; parse rules (PBITS, PBITS+1)
XRETURN         = $BE9E         ; a bare RTS in the global page (chain default)
GETBUFR         = $BEF5         ; reserve A pages; BI lowers HIMEM + registers it
                                ; (unusable on ProDOS 2.4.3's BASIC.SYSTEM for a
                                ; fixed-ORG resident - returned page != $80)
FREEBUFR        = $BEF8         ; release the space taken by GETBUFR
RSHIMEM         = $BEFB         ; BI's cached HIMEM page: restored into MEMSIZ+1
                                ; whenever the last file buffer is freed

;-------------------------------------------------------------------------
; BASIC.SYSTEM error codes (Table 5-6 in "Advanced Features for Programmers")
;-------------------------------------------------------------------------
BE_RANGE        = $02           ; RANGE ERROR   (bad device #, out of range)
BE_NODEV        = $03           ; NO DEVICE CONNECTED
BE_WRITEPROT    = $04
```

```
BE_PATHNF       = $06           ; PATH NOT FOUND
BE_EOD          = $05           ; END OF DATA
BE_IOERROR      = $08           ; I/O ERROR
BE_BADPARM      = $0B           ; INVALID PARAMETER
BE_TYPEMISMATCH = $0D           ; FILE TYPE MISMATCH
BE_TOOLARGE     = $0E           ; PROGRAM TOO LARGE
BE_SYNTAX       = $10           ; SYNTAX ERROR

;--------------------------------------------------------------------------
; SmartPort call numbers / status codes
;--------------------------------------------------------------------------
SP_CMD_STATUS   = $00
SP_CMD_READBLK  = $01
SP_CMD_WRITEBLK = $02
SP_CMD_FORMAT   = $03
SP_CMD_CONTROL  = $04
SP_CMD_INIT     = $05
SP_CMD_OPEN     = $06
SP_CMD_CLOSE    = $07
SP_CMD_READ     = $08
SP_CMD_WRITE    = $09

SP_STATUS_CODE  = $00           ; device status
SP_STATUS_DIB   = $03           ; device information block (name, type)

;--------------------------------------------------------------------------
; FujiNet network channel status codes (the "err" byte of a STATUS reply).
; Values from fujinet-firmware lib/network-protocol/status_error_codes.h.
;--------------------------------------------------------------------------
NERR_SUCCESS    = 1             ; all is well
NERR_EOF        = 136           ; end of data (a fine state, not a failure)
NERR_FNF        = 170           ; file not found

;--------------------------------------------------------------------------
; FujiNet network command bytes (SmartPort CONTROL/STATUS codes).
; Values confirmed against fujinet-firmware include/fujiCommandID.h.
;--------------------------------------------------------------------------
NETCMD_OPEN         = 'O'       ; $4F open: mode, trans, url
NETCMD_CLOSE        = 'C'       ; $43 close
NETCMD_READ         = 'R'       ; $52 read (via STATUS)
NETCMD_WRITE        = 'W'       ; $57 write
NETCMD_STATUS       = 'S'       ; $53 status (bytes waiting / connected / error)
NETCMD_PARSE        = 'P'       ; $50 JSON parse
NETCMD_QUERY        = 'Q'       ; $51 JSON query
NETCMD_ACCEPT       = 'A'       ; $41 TCP accept (NETCMD_CONTROL)
NETCMD_CHDIR        = ','       ; $2C NCD - change directory / set prefix
NETCMD_MKDIR        = '*'       ; $2A NMKDIR
NETCMD_RMDIR        = '+'       ; $2B NRMDIR
NETCMD_DELETE       = '!'       ; $21 NDEL
NETCMD_GETCWD       = '0'       ; $30 get current prefix
NETCMD_HTTPMODE     = 'M'       ; $4D HTTP channel mode (NETCMD_SET_CHANNEL_MODE)
NETCMD_CHANNEL_MODE = $FC       ; channel mode (0=protocol, 1=JSON)
NETCMD_USERNAME     = $FD       ; set username for next open
NETCMD_PASSWORD     = $FE       ; set password for next open
NETCMD_SET_CHANNEL  = $FA       ; select the active network channel/unit

;--------------------------------------------------------------------------
; FujiNet open modes / translation (match Adam SmartBASIC semantics)
;--------------------------------------------------------------------------
MODE_READ       = 4
MODE_WRITE      = 8
MODE_RDWR       = 12
MODE_POST       = 13
MODE_DIR        = 6             ; directory open mode (NCAT/NCATALOG)

; Directory listing formats, passed as the open translation byte (TRANS) and
; interpreted by the firmware's DIR_FORMAT.  NCAT asks for the 40-column ProDOS
; CAT layout; NCATALOG the 80-column CATALOG layout.
DIRFMT_CAT      = $84           ; DIR_FORMAT::A2CAT
DIRFMT_CATALOG  = $85           ; DIR_FORMAT::A2CATALOG

;--------------------------------------------------------------------------
; Scratch channel for the devicespec-only filesystem commands (NCD/NCAT/NMKDIR/
```

```
; NRMDIR/NDEL/NLOAD/NSAVE).  User channels are 1..15 (see CHKCHAN); channel 0
; can never be a user-opened channel, so filesystem ops here never clobber a
; live connection.
;-------------------------------------------------------------------------
SCRATCH_CHAN    = 0
```

fujiapple.s — install, recognizer, and the twenty commands

```
;*************************************************************************
; fujiapple.s - FujiNet BASIC.SYSTEM extension for Apple II / ProDOS 8
;
; A from-scratch rewrite of the FujiNet Applesoft extension.  It installs 17
; network commands (matching the Adam SmartBASIC 1.x FujiNet set) as
; BASIC.SYSTEM *external commands* rather than ampersand routines.
;
; Why external commands instead of "&"?  The "&" vector runs AFTER Applesoft's
; ROM tokenizer has crunched the line, so command names containing embedded
; keywords (NREAD->READ, NSTATUS->AT, NLOAD->LOAD, NJSONPARSE->ON, ...) get
; partially tokenized and must be matched byte-for-byte against ROM-crunched
; forms.  That fragility is the source of the original's subtle bugs.  A
; BASIC.SYSTEM external command instead receives the RAW, untokenized command
; line at $200 - the tokenizer never touches it.
;
; Residency model: the whole image loads at RESSTART ($8000) via BRUN and stays
; put; installation lowers HIMEM to a full 1K below the code (see BUFGUARD),
; syncs BI's cached HIMEM page (RSHIMEM), and hooks EXTRNCMD.  No runtime
; relocation (and thus no relocation-table rot) is needed.
;
; Invocation:
;   ] NOPEN 1,"N:HTTP://...",4,0        (immediate mode, typed bare)
;   100 PRINT CHR$(4);"NOPEN 1,...":..  (in a program, standard ProDOS idiom)
;*************************************************************************

        .include "equ.inc"

RESSTART = $8000                ; load / resident base (see fn-ext.cfg)

; BASIC.SYSTEM puts the FIRST open file's 1K buffer at [HIMEM, HIMEM+$400) -
; ABOVE HIMEM (that's what the stock $9600-$99FF gap under BI is for).  So
; HIMEM must sit a full 1K below the resident, or the first CATALOG/OPEN
; shreds our code with a directory block (whose leading $00 00 becomes a BRK
; right at CMDHANDLER - the "NEW after CATALOG" monitor break).
BUFGUARD = $0400
HIMEMTOP = RESSTART - BUFGUARD  ; $7C00: what we set MEMSIZ/FRETOP/RSHIMEM to

; Zero-page pointers we borrow for indirect addressing (must be in ZP).
TBLP     = ZP1                  ; keyword-table walk pointer ($EB/$EC)
STRP     = ZP3                  ; PUTS string pointer        ($FA/$FB)
SLOTPTR  = ZP2                  ; SmartPort slot scan ptr    ($ED/$EE)
GUSRC    = ZP4                  ; GETURL source string ptr   ($FC/$FD)

        .segment "CODE"

;=========================================================================
; Fixed entry table at the resident base (RESSTART).  BRUN runs the first slot;
; the others are stable CALL targets independent of code layout:
;   CALL 32768 ($8000) - (BRUN) install
;   CALL 32771 ($8003) - printer redirect ON  (COUT -> FujiNet PRINTER)
;   CALL 32774 ($8006) - printer redirect OFF (restore previous output)
;   CALL 32777 ($8009) - INTERNAL: error-trap recovery (see CMDENTRY).  It is
;                        reached only via the trap's fake program line; calling
;                        it by hand would restore a stale stack pointer.
;=========================================================================
ENTRY:
        JMP INSTALL
        JMP PRON
```

```
        JMP PROFF
        JMP RECOVER

;===============================================================================
; INSTALL - runs once when the BIN is BRUN.  Protects the resident with HIMEM
; and chains our recognizer into the BASIC.SYSTEM external-command vector.
;===============================================================================
INSTALL:
        ; --- verify there is room below the current HIMEM for the resident ---
        LDA MEMSIZ+1            ; current HIMEM hi
        CMP #>RES_END
        BCC @toolow             ; HIMEM hi < RES_END hi  -> no room
        BNE @room               ; HIMEM hi > RES_END hi  -> ok
        LDA MEMSIZ              ; equal hi: compare lo
        CMP #<RES_END
        BCS @room
@toolow:
        JMP @noroom             ; (out of BCC range; hop via a near JMP)
@room:
        ; --- claim the resident: HIMEM goes to HIMEMTOP ($7C00), a full 1K
        ; BELOW the code, because BI parks the first open file's buffer in the
        ; 1K ABOVE HIMEM (see BUFGUARD above).  Also sync BI's cached HIMEM
        ; page (RSHIMEM): BI allocates further buffers by subtracting pages
        ; from live MEMSIZ, but when the last buffer closes it restores MEMSIZ
        ; from RSHIMEM and reorganizes - a mismatch there re-exposes the
        ; resident.  With all three in sync, every buffer lands below $8000
        ; and the close-time restore is a no-op.  (Behavior disassembled from
        ; ProDOS 2.4.3's BASIC.SYSTEM; its GETBUFR vector proved unusable for
        ; a fixed-ORG resident.)
        LDA #<HIMEMTOP
        STA MEMSIZ
        STA FRETOP
        LDA #>HIMEMTOP
        STA MEMSIZ+1
        STA FRETOP+1
        STA RSHIMEM

        LDA #0
        STA CURTRANS            ; default translation = none (NTRANS/NTYPE)

        ; --- chain into EXTRNCMD ($BE06 = JMP; target @ $BE07/$BE08) ---
        LDA EXTRNCMD+1          ; save the previous handler so we can daisy-chain
        STA NEXTCMD+1
        LDA EXTRNCMD+2
        STA NEXTCMD+2
        LDA #<CMDHANDLER
        STA EXTRNCMD+1
        LDA #>CMDHANDLER
        STA EXTRNCMD+2

        JSR PRINTBANNER

        ; --- locate the SmartPort card and the FujiNet "NETWORK" device ---
        JSR FINDSP
        BCS @nosp
        ; INSTALL runs from inside the STARTUP program (via BRUN).  The SmartPort
        ; device scan below scribbles Applesoft zero page (the emulator's host
        ; firmware uses it as scratch), which would corrupt STARTUP's execution
        ; state and later crash the first DOS command + NEW.  Snapshot all of
        ; zero page around the scan and restore it (results land in SPUNIT/
        ; SPPUNIT in BSS, so this is safe; unlike the per-command path there is
        ; no interleaved Applesoft parsing here to disturb).  ARGBUF is free at
        ; install time, so it doubles as the snapshot buffer.
        LDX #0
@svzp:  LDA $00,X
        STA ARGBUF,X
        INX
        BNE @svzp
        JSR FINDNET
        JSR FINDPRINTER         ; optional; SPPUNIT stays 0 if there is no printer
        LDX #0
@rszp:  LDA ARGBUF,X
        STA $00,X
```

```
        INX
        BNE @rszp
        LDA SPUNIT
        BEQ @nonet
        LDA #<MSG_NETOK         ; "FUJINET NETWORK DEVICE AT SP UNIT $"
        LDY #>MSG_NETOK
        JSR PUTS
        LDA SPUNIT
        JSR PUTHEX
        JSR CROUT
        LDA SPPUNIT             ; printer found?
        BEQ @noprint
        LDA VECOUT              ; sane PROFF default even if PRON never ran
        STA SAVVEC
        LDA VECOUT+1
        STA SAVVEC+1
        LDA #<MSG_PROK
        LDY #>MSG_PROK
        JSR PUTS
        LDA SPPUNIT
        JSR PUTHEX
        JSR CROUT
@noprint:
        RTS
@nosp:
        LDA #<MSG_NOSP
        LDY #>MSG_NOSP
        JSR PUTS
        RTS
@nonet:
        LDA #<MSG_NONET
        LDY #>MSG_NONET
        JSR PUTS
        RTS

@noroom:
        LDA #<MSG_NOROOM
        LDY #>MSG_NOROOM
        JSR PUTS
        RTS

;================================================================================
; CMDHANDLER - installed at EXTRNCMD.  When BASIC.SYSTEM's internal command
; table fails to match a typed line, it does SEC and tail-JMPs into $BE06, so
; our RTS returns straight to BI's caller and CARRY is the claim flag.  The
; raw command line is at $200, ASCII with the high bit set.
;
;   match one of our keywords  -> set up the parse vectors, return CLC
;   no match                   -> SEC (decline - MANDATORY, see @nomatch) and
;                                 daisy-chain to the previous handler (NEXTCMD)
;================================================================================
CMDHANDLER:
        CLD                     ; identification byte for future BASIC.SYSTEMs

        ; Find where the command keyword starts (skip leading blanks).
        LDX #0
@skipsp:
        LDA IN,X
        CMP #SPACE              ; space with high bit set
        BNE @scan
        INX
        BNE @skipsp

@scan:
        ; X = index in $200 of first non-blank char.  Walk the keyword table.
        STX CMDSTART
        LDA #<CMDTABLE
        STA TBLP
        LDA #>CMDTABLE
        STA TBLP+1

@nextcmd:
        LDY #0                  ; Y walks the table entry's name
```

```
        LDX CMDSTART            ; X walks the input line
        LDA (TBLP),Y            ; first name byte
        BEQ @nomatch            ; $00 first byte -> end of table

@cmp:
        LDA (TBLP),Y
        BEQ @endname            ; reached the NUL that ends this name
        EOR IN,X                ; compare (table = plain ASCII, input = hi-bit)
        AND #$7F                ; ignore the input's high bit
        BNE @advance            ; mismatch -> try next table entry
        INY
        INX
        BNE @cmp

@endname:
        ; Keyword characters all matched.  Require a delimiter next so that
        ; e.g. "NCD" does not swallow a longer word.
        LDA IN,X
        AND #$7F
        CMP #' '+1              ; space, CR, NUL, anything <= space -> delimiter
        BCC @matched
        CMP #','
        BEQ @matched
        CMP #'"'
        BEQ @matched
        ; not a delimiter -> not really our command
@advance:
        ; skip to the byte after this entry's NUL, then past its 2 addr bytes
        LDY #0
@toz:
        LDA (TBLP),Y
        INY                     ; Y advances; A keeps the byte just read
        CMP #0
        BNE @toz                ; keep going until we consumed the NUL
        ; Y now points just past the NUL; skip the 2 handler-address bytes
        INY
        INY
        TYA
        CLC
        ADC TBLP
        STA TBLP
        BCC @nextcmd
        INC TBLP+1
        BNE @nextcmd            ; (always)

@matched:
        ; X indexes the first argument character in $200.  Remember it.
        STX ARGSTART
        ; Y indexes the table NUL; the 2 bytes after it are the handler addr.
        ; Stash the real handler in HANDLERVEC and point BASIC.SYSTEM at the
        ; CMDENTRY trampoline instead, so every command's Applesoft text state
        ; is saved/restored around it (see CMDENTRY).
        INY
        LDA (TBLP),Y
        STA HANDLERVEC
        INY
        LDA (TBLP),Y
        STA HANDLERVEC+1
        LDA #<CMDENTRY
        STA XTRNADDR
        LDA #>CMDENTRY
        STA XTRNADDR+1

        ; XLEN = keyword length - 1 = (ARGSTART - CMDSTART) - 1
        SEC
        LDA ARGSTART
        SBC CMDSTART
        SEC
        SBC #1
        STA XLEN
        LDA #0
        STA XCNUM               ; external command number
        STA PBITS               ; PBITS = 0  -> BASIC.SYSTEM does no parsing
```

```
        STA PBITS+1
        CLC                     ; claimed, no error
        RTS

@nomatch:
        ; Not ours.  The protocol REQUIRES carry SET on decline; our table walk
        ; reaches here with carry CLEAR (the final CLC/ADC entry advance), which
        ; told BASIC.SYSTEM we CLAIMED the line.  It then dispatched through the
        ; STALE XTRNADDR/XLEN/PBITS of the previous command - so after any
        ; N-command, every typed line we don't recognize (NEW, CALL -151, ...)
        ; re-entered the old handler against a garbage buffer and crashed inside
        ; the keyboard wedge.  SEC = decline, cleanly.
        SEC
        JMP NEXTCMD             ; hand off to the previously installed handler

; NEXTCMD's operand is patched at install time to the prior EXTRNCMD target
; (defaults to XRETURN, the global page's do-nothing RTS).
NEXTCMD:
        JMP XRETURN

;===============================================================================
; CMDENTRY - single dispatch trampoline for every command.  BASIC.SYSTEM
; JSRs here (via XTRNADDR) after CMDHANDLER claims a keyword.
;
; It does two jobs:
;
; 1. TEXT STATE.  Our handlers repoint Applesoft's TXTPTR into ARGBUF (via
;    SETARGS) so the ROM evaluators can parse the argument tail.  BASIC.SYSTEM
;    does not save TXTPTR/CURLIN for us, so snapshot them here and restore on
;    the way out, making us behaviourally identical to a well-behaved external
;    command.  The handler's return status (carry, A) survives the restore.
;
; 2. THE ERROR TRAP.  The ROM evaluators bail out through Applesoft's ERROR
;    on bad input (SYNERR from a misplaced argument, type mismatch, ...).
;    That path never returns to BASIC.SYSTEM's dispatch, leaving its command
;    wedge wedged: afterwards every command line - ours AND the built-ins -
;    falls through to Applesoft as ?SYNTAX ERROR until reboot.  So run the
;    handler under a fake ONERR frame (flow verified from the ROM, see
;    equ.inc): ERROR with ERRFLG set jumps to HANDLERR, which loads TXTPTR
;    from ONRTXT and CURLIN from ONRLIN, then GOTOs the line number found at
;    TXTPTR, searching the program at TXTTAB.  We point ONRTXT at the text
;    "0" and TXTTAB at FAKEPROG, a one-line program reading "0 CALL 32777" -
;    whose target is RECOVER below.  RECOVER unwinds the stack to the level
;    recorded in TRAPSP and exits through CMDFAIL: the command fails with a
;    clean, ONERR-catchable SYNTAX ERROR and BASIC.SYSTEM lives on.
;
;    Because TXTTAB is swapped during the handler, NLOAD/NSAVE/RELINK read
;    the real value from SAVTTAB instead.
;===============================================================================
CMDENTRY:
        LDA TXTPTR              ; --- snapshot Applesoft text state ---
        STA SAVTXT
        LDA TXTPTR+1
        STA SAVTXT+1
        LDA CURLIN
        STA SAVCURLIN
        LDA CURLIN+1
        STA SAVCURLIN+1
        LDA ERRFLG              ; --- snapshot the user's ONERR state ---
        STA SAVONERR
        LDA ONRTXT
        STA SAVONERR+1
        LDA ONRTXT+1
        STA SAVONERR+2
        LDA ONRLIN
        STA SAVONERR+3
        LDA ONRLIN+1
        STA SAVONERR+4
        LDA TRCFLG              ; BI parks $A5 here during command processing;
        STA SAVONERR+5          ; bit 7 reads as TRACE and would print "#0"
        LDA #0                  ; when the trap's decoy line runs.  Silence it
        STA TRCFLG              ; for the handler's duration.
        LDA TXTTAB
```

```
        STA SAVTTAB
        LDA TXTTAB+1
        STA SAVTTAB+1
        LDA #<FAKETGT           ; --- arm the trap ---
        STA ONRTXT
        LDA #>FAKETGT
        STA ONRTXT+1
        LDA #0
        STA ONRLIN              ; CURLIN 0 makes GOTO search from TXTTAB
        STA ONRLIN+1
        LDA #<FAKEPROG
        STA TXTTAB
        LDA #>FAKEPROG
        STA TXTTAB+1
        LDA #$80
        STA ERRFLG
        TSX                     ; stack level RECOVER unwinds to
        STX TRAPSP
        JSR CMDCALL             ; run the real handler; RTS returns here
        PHP                     ; preserve error/carry + A across the restore
        PHA
        JMP CMDRESTOR
CMDFAIL:
        ; Entered from RECOVER after a trapped ROM error: the stack is back
        ; at CMDENTRY's level, so exiting through the common restore returns
        ; to BASIC.SYSTEM exactly like a normal failed command.
        LDA #BE_SYNTAX
        SEC
        PHP
        PHA
CMDRESTOR:
        LDA SAVTXT              ; --- restore text state ---
        STA TXTPTR
        LDA SAVTXT+1
        STA TXTPTR+1
        LDA SAVCURLIN
        STA CURLIN
        LDA SAVCURLIN+1
        STA CURLIN+1
        LDA SAVONERR            ; --- disarm: restore the user's ONERR ---
        STA ERRFLG
        LDA SAVONERR+1
        STA ONRTXT
        LDA SAVONERR+2
        STA ONRTXT+1
        LDA SAVONERR+3
        STA ONRLIN
        LDA SAVONERR+4
        STA ONRLIN+1
        LDA SAVONERR+5          ; give BI its $F2 sentinel back
        STA TRCFLG
        LDA SAVTTAB
        STA TXTTAB
        LDA SAVTTAB+1
        STA TXTTAB+1
        PLA
        PLP
        RTS
CMDCALL:
        JMP (HANDLERVEC)

; RECOVER - the target of the trap's "0 CALL 32777" line.  An Applesoft error
; fired inside a handler; the ROM has already vectored through HANDLERR ->
; GOTO -> NEWSTT -> CALL to get here.  Throw away everything above CMDENTRY's
; recorded stack level and fail the command cleanly.
RECOVER:
        LDX TRAPSP
        TXS
        JMP CMDFAIL

;===============================================================================
; SETARGS - point Applesoft's TXTPTR at a clean copy of the argument tail so
; the ROM evaluators (GETBYT / CHKCOM / FRMNUM / PTRGET) can parse it.
```

```
;
; The line at $200 has the high bit set on every byte; the ROM routines expect
; plain ASCII.  Copy from $200+ARGSTART up to the CR into ARGBUF with the high
; bits stripped and a $00 terminator, then set TXTPTR = ARGBUF-1 (CHRGET
; pre-increments).  Every command handler calls this first.
;================================================================================
SETARGS:
        LDX ARGSTART
        LDY #0
@cp:
        LDA IN,X
        AND #$7F
        CMP #CR
        BEQ @done
        STA ARGBUF,Y
        INX
        INY
        CPY #250
        BCC @cp
@done:
        LDA #0
        STA ARGBUF,Y            ; NUL terminate
        LDA #<(ARGBUF-1)
        STA TXTPTR
        LDA #>(ARGBUF-1)
        STA TXTPTR+1
        ; Prime the pump: the ROM evaluators (FRMNUM/GETBYT/PTRGET/CHKCOM) work
        ; on the CURRENT char via CHRGOT - they do NOT CHRGET first.  Set
        ; TXTPTR = ARGBUF-1 above, then CHRGET once to advance onto ARGBUF[0]
        ; (skipping leading spaces) so CHRGOT sees the first real arg character.
        ; Without this, FRMNUM reads the uninitialized byte before ARGBUF and
        ; dies with ?SYNTAX ERROR before ever looking at the argument.
        JSR CHRGET
        RTS

;================================================================================
; Command handlers (second phase).  BASIC.SYSTEM reaches these via the CMDENTRY
; trampoline (which saves/restores Applesoft's TXTPTR/CURLIN) after a successful
; claim.  Each must return: CLC + A=0 on success, or SEC + A=error code (a
; BASIC.SYSTEM code) to raise an ONERR-catchable error.
;================================================================================
;--- NOPEN d, url$, mode, trans -----------------------------------------
H_NOPEN:
        JSR CHKDEV
        BCS @err
        JSR SETARGS
        JSR GETBYT
        JSR CHKCHAN             ; -> CHAN (1..15) or SEC + A=range error
        BCS @err
        LDA CHAN
        JSR FN_SETCH            ; select channel once
        JSR CHKCOM
        JSR GETURL              ; url$ -> URLBUF / URLLEN
        JSR CHKCOM
        JSR GETBYT
        STX MODE
        JSR CHKCOM
        JSR GETBYT
        STX TRANS
        JSR ISHTTP
        BCS @http
        JSR OPENCK              ; verify the open; PATH NOT FOUND / I/O ERROR
        BCS @err
        CLC
        LDA #0
        RTS
@http:
        JSR FN_OPEN             ; HTTP(S): a status here would fire the request
        CLC                     ; early (breaking NHTTPMODE header setup), so
        LDA #0                  ; open errors surface via NSTATUS instead
@err:   RTS

;--- NCLOSE d -----------------------------------------------------------
```

```
H_NCLOSE:
        JSR CHKDEV
        BCS @err
        JSR SETARGS
        JSR GETBYT
        JSR CHKCHAN
        BCS @err
        LDA CHAN
        JSR FN_SETCH
        JSR FN_CLOSE
        CLC
        LDA #0
@err:   RTS

;--- NSTATUS d, bw, conn, err ------------------------------------------
H_NSTATUS:
        JSR CHKDEV
        BCS @err
        JSR SETARGS
        JSR GETBYT
        JSR CHKCHAN
        BCS @err
        LDA CHAN
        JSR FN_SETCH
        JSR FN_STATUS           ; SP_PAYLOAD = [avail_lo][avail_hi][conn][err]
        JSR CHKCOM
        LDA SP_PAYLOAD+1        ; bytes-waiting hi
        LDY SP_PAYLOAD          ; bytes-waiting lo
        JSR WORDSAV             ; store word into bw variable
        JSR CHKCOM
        LDY SP_PAYLOAD+2        ; connected
        JSR BYTESAV
        JSR CHKCOM
        LDY SP_PAYLOAD+3        ; device error
        JSR BYTESAV
        CLC
        LDA #0
@err:   RTS

;--- NREAD d, buf$, len ------------------------------------------------
H_NREAD:
        JSR CHKDEV
        BCS @err
        JSR SETARGS
        JSR GETBYT
        JSR CHKCHAN
        BCS @err
        LDA CHAN
        JSR FN_SETCH
        JSR CHKCOM
        JSR PTRGET              ; locate buf$ ; descriptor addr -> VARPTR
        LDA VARPTR
        STA RDVAR
        LDA VARPTR+1
        STA RDVAR+1
        JSR CHKCOM
        JSR GETBYT
        STX BUFLEN              ; requested length (0..255)
        LDA BUFLEN
        JSR FN_READ             ; SP_COUNT = actual bytes, data in SP_PAYLOAD
        JSR STORESTR            ; buf$ = exactly the bytes actually read
        CLC
        LDA #0
@err:   RTS

;--- NWRITE d, buf$, len -----------------------------------------------
H_NWRITE:
        JSR CHKDEV
        BCS @err
        JSR SETARGS
        JSR GETBYT
        JSR CHKCHAN
        BCS @err
```

```
        LDA CHAN
        JSR FN_SETCH            ; select channel BEFORE staging data
        JSR CHKCOM
        JSR GETURL              ; buf$ -> URLBUF / URLLEN
        JSR CHKCOM
        JSR GETBYT
        STX BUFLEN              ; requested length
        LDA BUFLEN              ; clamp to LEN(buf$)
        CMP URLLEN
        BCC @wlen
        LDA URLLEN
        STA BUFLEN
@wlen:
        LDX #0                  ; stage buf$ bytes -> SP_PAYLOAD
@wcp:
        CPX BUFLEN
        BEQ @wgo
        LDA URLBUF,X
        STA SP_PAYLOAD,X
        INX
        BNE @wcp
@wgo:
        LDA BUFLEN
        JSR FN_WRITE
        CLC
        LDA #0
@err:   RTS
;--- NJSONPARSE d ------------------------------------------------------
; Put channel d into JSON mode and parse the JSON document it is reading, so a
; following NJSONQUERY can pull values out of it.
H_NJSONPARSE:
        JSR CHKDEV
        BCC @jp1
        RTS
@jp1:
        JSR SETARGS
        JSR GETBYT
        JSR CHKCHAN
        BCC @jp2
        RTS
@jp2:
        LDA CHAN
        JSR FN_SETCH
        LDA #1                  ; channel mode -> JSON: data_buffer[0]=1
        STA SP_PAYLOAD+2
        LDA #1
        STA SP_PAYLOAD          ; length = 1
        LDA #0
        STA SP_PAYLOAD+1
        LDX SPUNIT
        LDY #NETCMD_CHANNEL_MODE
        JSR SP_CONTROL
        LDA #0                  ; parse: empty control list
        STA SP_PAYLOAD
        STA SP_PAYLOAD+1
        LDX SPUNIT
        LDY #NETCMD_PARSE
        JSR SP_CONTROL
        JMP SPRESULT

;--- NJSONQUERY d, result$, query$ -------------------------------------
; Send a JSONPath-ish query on channel d (already parsed via NJSONPARSE) and
; read the resulting value into result$.
H_NJSONQUERY:
        JSR CHKDEV
        BCC @jq1
        RTS
@jq1:
        JSR SETARGS
        JSR GETBYT
        JSR CHKCHAN
        BCC @jq2
        RTS
```

```
@jq2:
        LDA CHAN
        JSR FN_SETCH
        JSR CHKCOM
        JSR PTRGET              ; locate result$ ; descriptor addr -> VARPTR
        LDA VARPTR
        STA RDVAR
        LDA VARPTR+1
        STA RDVAR+1
        JSR CHKCOM
        JSR GETURL              ; query$ -> URLBUF / URLLEN
        LDX #0                  ; stage the query (no NUL, like the original)
@qcp:   CPX URLLEN
        BEQ @qgo
        LDA URLBUF,X
        STA SP_PAYLOAD+2,X
        INX
        BNE @qcp
@qgo:
        STX SP_PAYLOAD          ; count = URLLEN
        LDA #0
        STA SP_PAYLOAD+1
        LDX SPUNIT
        LDY #NETCMD_QUERY
        JSR SP_CONTROL
        LDA #16                 ; bounded poll for the value to become available
        STA JQRETRY
@poll:
        JSR FN_STATUS           ; SP_PAYLOAD = [avail_lo][avail_hi][conn][err]
        LDA SP_PAYLOAD
        ORA SP_PAYLOAD+1
        BNE @have
        DEC JQRETRY
        BNE @poll
        LDA #0                  ; timed out -> empty result$
        STA SP_COUNT
        STA SP_COUNT+1
        JSR STORESTR
        CLC
        LDA #0
        RTS
@have:
        LDA SP_PAYLOAD          ; read min(avail,255) bytes
        LDX SP_PAYLOAD+1
        BEQ @rd
        LDA #255
@rd:    JSR FN_READ             ; -> SP_PAYLOAD, SP_COUNT
        JSR STORESTR            ; result$ = the value bytes
        CLC
        LDA #0
        RTS
;--- devicespec-only filesystem commands: NCD/NMKDIR/NRMDIR/NDEL ---------------
; Each takes a single devicespec and issues one FujiNet CONTROL command against
; the scratch channel; the firmware parses the devicespec fresh (no OPEN needed).
H_NCD:          LDY #NETCMD_CHDIR       ; set prefix / change directory
                JMP NCTL
H_NMKDIR:       LDY #NETCMD_MKDIR
                JMP NCTL
H_NRMDIR:       LDY #NETCMD_RMDIR
                JMP NCTL
H_NDEL:         LDY #NETCMD_DELETE
                JMP NCTL

; NCTL - common body.  Y = FujiNet control code.  Parse the devicespec, select
; the scratch channel, send the control, and surface a device error as I/O ERROR.
NCTL:
        STY NCTLCMD             ; GETURL clobbers Y, so stash the control code
        JSR CHKDEV
        BCS @err
        JSR SETARGS
        JSR GETURL              ; devicespec -> URLBUF / URLLEN
        LDA #SCRATCH_CHAN
        JSR FN_SETCH
```

```
        LDY NCTLCMD
        JSR FN_CONTROL_DS       ; returns carry = SmartPort/device error
        JMP SPRESULT
@err:   RTS

;--- NCAT / NCATALOG devicespec ------------------------------------------
; Open the devicespec as a directory (mode 6) and print the ProDOS-style listing
; the firmware formats for us: NCAT requests the 40-column CAT layout, NCATALOG
; the 80-column CATALOG layout (selected via the TRANS byte).  Read and print
; the listing a chunk at a time until no bytes remain, then close.  Prints
; straight to the screen (no string variable), so it doubles as an end-to-end
; read-path test.
H_NCAT:
        LDA #DIRFMT_CAT
        BNE NCATCOM             ; (always: DIRFMT_CAT != 0)
H_NCATALOG:
        LDA #DIRFMT_CATALOG
NCATCOM:
        STA TRANS               ; survives the parse below (only dir handlers write TRANS)
        JSR CHKDEV
        BCS @err
        JSR SETARGS
        JSR GETURL              ; devicespec -> URLBUF / URLLEN
        LDA #SCRATCH_CHAN
        JSR FN_SETCH
        LDA #MODE_DIR
        STA MODE
        JSR OPENCK              ; bad path -> PATH NOT FOUND instead of silence
        BCS @err
@loop:
        JSR FN_STATUS           ; SP_PAYLOAD = [avail_lo][avail_hi][conn][err]
        LDA SP_PAYLOAD
        ORA SP_PAYLOAD+1
        BEQ @done               ; nothing waiting -> end of listing
        LDA SP_PAYLOAD+1        ; >= 256 waiting?
        BNE @full
        LDA SP_PAYLOAD          ; else read the (1..255) available
        BNE @rd                 ; (always: the ORA above proved lo|hi nonzero)
@full:  LDA #255
@rd:    JSR FN_READ             ; A bytes -> SP_PAYLOAD, SP_COUNT = actual
        LDA SP_COUNT
        ORA SP_COUNT+1
        BEQ @done               ; defensive: read returned nothing
        LDX #0
@pr:    CPX SP_COUNT
        BEQ @loop
        LDA SP_PAYLOAD,X
        CMP #$0A                ; the firmware ends each entry with CR+LF, but
        BEQ @next               ; COUT treats LF as a line feed too -> blank
        ORA #$80                ; lines.  Drop the LF; let the CR do the newline.
        JSR COUT
@next:  INX
        BNE @pr
        BEQ @loop               ; (always)
@done:
        JSR FN_CLOSE
        CLC
        LDA #0
@err:   RTS

;--- NTYPE devicespec ----------------------------------------------------
; Open the devicespec on channel 1 (mode READ) using the current NTRANS
; translation, print its contents to the screen a chunk at a time, and close.
; Structurally NCAT with MODE_READ and a user-set TRANS; output is faithful
; (each byte COUT'd with the high bit set, no LF-dropping) - line-ending
; handling is the caller's choice via NTRANS.
H_NTYPE:
        JSR CHKDEV
        BCS @err
        JSR SETARGS
        JSR GETURL              ; devicespec -> URLBUF / URLLEN
        LDA #1                  ; channel 1 (NTRANS/NTYPE operate on channel 1)
        JSR FN_SETCH
```

```
        LDA #MODE_READ
        STA MODE
        LDA CURTRANS
        STA TRANS
        JSR OPENCK              ; bad path -> PATH NOT FOUND / I/O ERROR
        BCS @err
@loop:
        JSR FN_STATUS           ; SP_PAYLOAD = [avail_lo][avail_hi][conn][err]
        JSR POLLKEY             ; CTRL-C aborts, CTRL-S pauses while we wait
        BCS @abort
        LDA SP_PAYLOAD
        ORA SP_PAYLOAD+1
        BNE @rd                 ; bytes waiting -> read them
        LDA SP_PAYLOAD+2        ; none waiting: still connected?
        BNE @loop               ; yes -> wait for more
        BEQ @done               ; no -> EOF
@rd:
        LDA SP_PAYLOAD+1        ; chunk = min(avail, 255)
        BEQ @lo
        LDA #255
        BNE @go
@lo:    LDA SP_PAYLOAD
@go:    JSR FN_READ             ; A bytes -> SP_PAYLOAD, SP_COUNT = actual
        LDA SP_COUNT
        ORA SP_COUNT+1
        BEQ @done               ; defensive: read returned nothing
        LDX #0
@pr:    CPX SP_COUNT
        BEQ @loop
        JSR POLLKEY             ; CTRL-C aborts, CTRL-S pauses the scroll
        BCS @abort
        LDA SP_PAYLOAD,X
        ORA #$80                ; COUT wants the high bit set
        JSR COUT
        INX
        BNE @pr
        BEQ @loop               ; (always)
@abort:
        JSR CROUT               ; end the partial line cleanly
        JSR FN_CLOSE            ; CTRL-C: close the channel and bail out
        CLC
        LDA #0
        RTS
@done:
        JSR FN_CLOSE
        CLC
        LDA #0
@err:   RTS
;--- NSAVE devicespec ------------------------------------------------------
; Write the tokenized Applesoft program (TXTTAB..VARTAB, including the $0000
; end marker) to the network file.  Reads program memory only - non-destructive.
H_NSAVE:
        JSR CHKDEV
        BCC @ns1
        RTS
@ns1:
        JSR SETARGS
        JSR GETURL              ; devicespec -> URLBUF / URLLEN
        LDA #SCRATCH_CHAN
        JSR FN_SETCH
        LDA #MODE_WRITE
        STA MODE
        LDA #0
        STA TRANS
        JSR OPENCK              ; unwritable target -> error, not silence
        BCC @nsok
        RTS
@nsok:
        LDA SAVTTAB             ; PRGPTR = TXTTAB (real value; the error trap
        STA GUSRC               ; has TXTTAB swapped - see CMDENTRY)
        LDA SAVTTAB+1
        STA GUSRC+1
@wloop:
```

```
        SEC                     ; remaining = VARTAB - PRGPTR
        LDA VARTAB
        SBC GUSRC
        STA SVLEN
        LDA VARTAB+1
        SBC GUSRC+1
        STA SVLEN+1
        LDA SVLEN
        ORA SVLEN+1
        BEQ @wdone
        LDA SVLEN+1             ; chunk = min(remaining, 255)
        BNE @wfull
        LDA SVLEN
        JMP @wchunk
@wfull: LDA #255
@wchunk:
        STA BUFLEN
        LDY #0                  ; copy chunk from (PRGPTR) into SP_PAYLOAD
@wcp:   CPY BUFLEN
        BEQ @wgo
        LDA (GUSRC),Y
        STA SP_PAYLOAD,Y
        INY
        BNE @wcp
@wgo:
        LDA BUFLEN
        JSR FN_WRITE
        LDA GUSRC               ; PRGPTR += chunk
        CLC
        ADC BUFLEN
        STA GUSRC
        BCC @wloop
        INC GUSRC+1
        JMP @wloop
@wdone:
        JSR FN_CLOSE
        CLC
        LDA #0
        RTS

;--- NLOAD devicespec ------------------------------------------------------
; Read a tokenized Applesoft program from the network into TXTTAB until EOF,
; then rebuild the line links (which also sets VARTAB) and reset the variable
; pointers so the loaded program LISTs and RUNs.
H_NLOAD:
        JSR CHKDEV
        BCC @nl1
        RTS
@nl1:
        JSR SETARGS
        JSR GETURL              ; devicespec -> URLBUF / URLLEN
        LDA #SCRATCH_CHAN
        JSR FN_SETCH
        LDA #MODE_READ
        STA MODE
        LDA #0
        STA TRANS
        JSR OPENCK              ; missing file -> PATH NOT FOUND, and the
        BCC @nlok               ; program in memory is left untouched
        RTS
@nlok:
        ; [TXTTAB-1] must be $00 for Applesoft.  (SAVTTAB = the real TXTTAB;
        ; the error trap has TXTTAB itself swapped - see CMDENTRY.)
        LDA SAVTTAB
        SEC
        SBC #1
        STA GUSRC
        LDA SAVTTAB+1
        SBC #0
        STA GUSRC+1
        LDY #0
        TYA
        STA (GUSRC),Y
```

```
        LDA SAVTTAB             ; PRGPTR = TXTTAB
        STA GUSRC
        LDA SAVTTAB+1
        STA GUSRC+1
@rloop:
        ; Bounds guard: the program must stay below HIMEM.  A chunk is at most
        ; 255 bytes, so stopping once the pointer reaches the HIMEMTOP page
        ; keeps every write below RESSTART (the worst case spills into the
        ; 1K buffer guard band, never the resident).
        LDA GUSRC+1
        CMP #>HIMEMTOP
        BCS @rtoobig
        JSR FN_STATUS           ; SP_PAYLOAD = [avail_lo][avail_hi][conn][err]
        LDA SP_PAYLOAD
        ORA SP_PAYLOAD+1
        BNE @rread
        LDA SP_PAYLOAD+2        ; no bytes: still connected?
        BNE @rloop              ; yes -> wait for more
        BEQ @rdone              ; no -> EOF
@rread:
        LDA SP_PAYLOAD+1        ; chunk = min(avail, 255)
        BEQ @rlo
        LDA #255
        BNE @rgo
@rlo:   LDA SP_PAYLOAD
@rgo:
        JSR FN_READ             ; -> SP_PAYLOAD, SP_COUNT
        LDY #0                  ; copy SP_COUNT bytes into (PRGPTR)
@rcp:   CPY SP_COUNT
        BEQ @radv
        LDA SP_PAYLOAD,Y
        STA (GUSRC),Y
        INY
        BNE @rcp
@radv:
        LDA GUSRC               ; PRGPTR += SP_COUNT
        CLC
        ADC SP_COUNT
        STA GUSRC
        BCC @rloop
        INC GUSRC+1
        JMP @rloop
@rdone:
        JSR FN_CLOSE
        JSR RELINK              ; rebuild links, set VARTAB
        JSR NLVARS              ; reset the variable pointers
        CLC
        LDA #0
        RTS
@rtoobig:
        ; File exceeds program space.  Close the channel, scrap the partial
        ; load (leave a clean empty program so LIST/RUN behave like after NEW),
        ; and raise PROGRAM TOO LARGE.
        JSR FN_CLOSE
        LDA SAVTTAB
        STA GUSRC
        LDA SAVTTAB+1
        STA GUSRC+1
        LDA #0
        TAY
        STA (GUSRC),Y           ; $0000 end marker at TXTTAB = empty program
        INY
        STA (GUSRC),Y
        JSR RELINK              ; VARTAB <- TXTTAB+2
        JSR NLVARS
        LDA #BE_TOOLARGE
        SEC
        RTS

; NLVARS - point the Applesoft variable-space pointers at a clean state after
; the program (or its scrapped remains) was replaced: no variables, no arrays,
; empty string pool.
NLVARS:
```

```
        LDA VARTAB              ; ARYTAB = STREND = VARTAB
        STA ARYTAB
        STA STREND
        LDA VARTAB+1
        STA ARYTAB+1
        STA STREND+1
        LDA MEMSIZ              ; FRETOP = HIMEM (empty string pool)
        STA FRETOP
        LDA MEMSIZ+1
        STA FRETOP+1
        RTS

;===============================================================================
; RELINK - rebuild the forward line links of the program at TXTTAB and set
; VARTAB to just past the $0000 end-of-program marker.  Re-scans each line for
; its $00 terminator, so it does not trust the (possibly stale) stored links.
; Applesoft line: [link lo][link hi][line# lo][line# hi][tokens...][$00];
; the program ends with a $0000 where the next link would be.
;===============================================================================
RELINK:
        LDA SAVTTAB             ; the real TXTTAB (see CMDENTRY's error trap)
        STA GUSRC
        LDA SAVTTAB+1
        STA GUSRC+1
@line:
        LDY #0
        LDA (GUSRC),Y           ; link lo
        INY
        ORA (GUSRC),Y           ; | link hi  -> both zero means end marker
        BNE @fix
        LDA GUSRC               ; VARTAB = PRGPTR + 2
        CLC
        ADC #2
        STA VARTAB
        LDA GUSRC+1
        ADC #0
        STA VARTAB+1
        RTS
@fix:
        LDY #4                  ; scan past link(2)+line#(2) for the $00
@scan:
        LDA (GUSRC),Y
        BEQ @eol
        INY
        BNE @scan
@eol:
        TYA                     ; NEXTPTR = PRGPTR + Y + 1
        SEC
        ADC GUSRC
        STA NEXTPTR
        LDA GUSRC+1
        ADC #0
        STA NEXTPTR+1
        LDY #0                  ; store NEXTPTR as this line's link
        LDA NEXTPTR
        STA (GUSRC),Y
        INY
        LDA NEXTPTR+1
        STA (GUSRC),Y
        LDA NEXTPTR             ; PRGPTR = NEXTPTR
        STA GUSRC
        LDA NEXTPTR+1
        STA GUSRC+1
        JMP @line
;--- NACCEPT d -----------------------------------------------------------------
; Accept an incoming connection on TCP listener channel d (opened with a
; "N:TCP://:port" devicespec).
H_NACCEPT:
        JSR CHKDEV
        BCC @ac1
        RTS
@ac1:
        JSR SETARGS
```

```
        JSR GETBYT
        JSR CHKCHAN
        BCC @ac2
        RTS
@ac2:
        LDA CHAN
        JSR FN_SETCH
        LDA #0                  ; empty control list
        STA SP_PAYLOAD
        STA SP_PAYLOAD+1
        LDX SPUNIT
        LDY #NETCMD_ACCEPT      ; 'A' -> accept_connection()
        JSR SP_CONTROL
        JMP SPRESULT
;--- NLOGIN d, user$, pass$ ------------------------------------------------
; Stash the username/password on channel d for its NEXT open (the firmware keeps
; them per-channel).  Sent as two control commands (USERNAME then PASSWORD).
H_NLOGIN:
        JSR CHKDEV
        BCS @lgerr
        JSR SETARGS
        JSR GETBYT
        JSR CHKCHAN
        BCS @lgerr
        LDA CHAN
        JSR FN_SETCH
        JSR CHKCOM
        JSR GETURL              ; user$ -> URLBUF / URLLEN
        LDY #NETCMD_USERNAME
        JSR FN_CONTROL_DS
        JSR CHKCOM
        JSR GETURL              ; pass$ -> URLBUF / URLLEN
        LDY #NETCMD_PASSWORD
        JSR FN_CONTROL_DS
        JMP SPRESULT
@lgerr: RTS

;--- NHTTPMODE d, mode -----------------------------------------------------
; Set the HTTP channel sub-mode on an already-open HTTP channel d.  mode:
;   0=BODY 1=COLLECT_HEADERS 2=GET_HEADERS 3=SET_HEADERS 4=SET_POST_DATA
; The firmware (process_http) reads the mode from data_buffer[1].
H_NHTTPMODE:
        JSR CHKDEV
        BCS @hmerr
        JSR SETARGS
        JSR GETBYT
        JSR CHKCHAN
        BCS @hmerr
        LDA CHAN
        JSR FN_SETCH
        JSR CHKCOM
        JSR GETBYT              ; mode -> X
        ; The firmware source reads the mode from data_buffer[1] (aux2), but the
        ; deployed binary behaves as if it reads data_buffer[0] (aux1).  Put the
        ; mode in BOTH so it works whichever slot the running firmware reads.
        STX SP_PAYLOAD+2        ; data_buffer[0] = aux1 = mode
        STX SP_PAYLOAD+3        ; data_buffer[1] = aux2 = mode
        LDA #0
        STA SP_PAYLOAD+4        ; pad
        LDA #3
        STA SP_PAYLOAD          ; control-list length = 3 bytes
        LDA #0
        STA SP_PAYLOAD+1
        LDX SPUNIT
        LDY #NETCMD_HTTPMODE
        JSR SP_CONTROL
        JMP SPRESULT
@hmerr: RTS

;--- NTRANS d, mode --------------------------------------------------------
; Record the current translation mode used by NTYPE (channel 1).  The firmware
; can only set translation at OPEN time (the IWM set-translation control is a
; stub), so this just stashes a byte for NTYPE's next open to consume - no
```

```
; SmartPort command is issued.  mode: 0=NONE 1=CR 2=LF 3=CR/LF 4=PETSCII.
; d is validated (1..15) for consistency; a single value is kept.
H_NTRANS:
        JSR CHKDEV
        BCS @err
        JSR SETARGS
        JSR GETBYT
        JSR CHKCHAN             ; validate channel 1..15
        BCS @err
        JSR CHKCOM
        JSR GETBYT              ; mode -> X
        STX CURTRANS
        CLC
        LDA #0
@err:   RTS

;================================================================================
; SPRESULT - turn the SmartPort carry (set on entry = device error) into a
; command return: CLC + A=0 on success, or SEC + A = I/O ERROR so ONERR can
; catch it.  Tail-called by the filesystem commands.
;================================================================================
SPRESULT:
        BCS @err
        CLC
        LDA #0
        RTS
@err:
        LDA #BE_IOERROR
        SEC
        RTS

;================================================================================
; STORESTR - assign the SP_COUNT bytes in SP_PAYLOAD to the string variable
; whose descriptor address was saved in RDVAR (by an earlier PTRGET).  Makes
; fresh string space of exactly SP_COUNT bytes and copies the data in.
; Used by NREAD and NJSONQUERY.
;================================================================================
STORESTR:
        LDA RDVAR
        STA VARPTR
        LDA RDVAR+1
        STA VARPTR+1
        LDA SP_COUNT
        JSR STRINI              ; make string space (len A); DSCTMP = descriptor
        LDY #0
        LDA DSCTMP
        STA (VARPTR),Y          ; length
        INY
        LDA DSCTMP+1
        STA (VARPTR),Y          ; addr lo
        INY
        LDA DSCTMP+2
        STA (VARPTR),Y          ; addr hi
        LDA SP_COUNT
        LDX #<SP_PAYLOAD
        LDY #>SP_PAYLOAD
        JMP MOVSTR              ; copy data into the new string space; ends in RTS

;================================================================================
; Printer redirect - route BASIC.SYSTEM console output to the FujiNet
; SmartPort PRINTER device.
;
; CALL 32771 = printer on, CALL 32774 = back to the screen.
;
; BI's I/O architecture (global page): CSW/KSW stay pointed at BI's wedge; BI
; delivers each character through its ACTIVE output vector VECOUT ($BE30,
; default COUT1).  PRON/PROFF swap VECOUT - the only hook point that works:
;   - hooking CSW directly does NOT survive (BI re-syncs CSW; the hook drops
;     out after a character - observed on hardware);
;   - PR#n and the PR#n,A<addr> address form both fail with NO DEVICE
;     CONNECTED: BI validates the vector before use and rejects anything that
;     is not a real slot ROM, so neither planting PRHOOK in the $BE10+2n slot
;     table nor the address form can select a RAM routine (ProDOS 2.4.3).
```

```
;
; PROFF also flushes a pending partial line (a PRINT ending in ";").
;===============================================================================
PRON:
        LDA SPPUNIT             ; no printer found at install time -> do nothing
        BNE @go
        RTS
@go:
        LDA #0
        STA PRLEN
        LDA VECOUT              ; remember the active output device for PROFF
        STA SAVVEC
        LDA VECOUT+1
        STA SAVVEC+1
        LDA #<PRHOOK            ; and make the printer the output device
        STA VECOUT
        LDA #>PRHOOK
        STA VECOUT+1
        RTS

PROFF:
        JSR PRFLUSH             ; push out any pending partial line
        LDA SAVVEC              ; restore the previous output device
        STA VECOUT
        LDA SAVVEC+1
        STA VECOUT+1
        RTS

; PRHOOK - the output DEVICE routine (installed in VECOUT by PRON).  A =
; character (high bit set).  Line-buffer it and flush to the printer on CR or
; at 80 columns.  Terminal: no screen echo, like a PR#-selected printer card.
; Preserves A/X/Y like COUT1.
PRHOOK:
        STA PRCH                ; stash the char
        TXA
        PHA
        TYA
        PHA
        LDA PRCH
        AND #$7F                ; printers want plain ASCII
        LDY PRLEN
        STA PRLINEBUF,Y
        INC PRLEN
        CMP #CR
        BEQ @flush
        LDA PRLEN
        CMP #80
        BCC @done
@flush:
        JSR PRFLUSH
@done:
        PLA
        TAY
        PLA
        TAX
        LDA PRCH                ; restore the char (COUT contract)
        RTS

; PRFLUSH - write the buffered line (PRLEN bytes) to the PRINTER unit, then
; reset the buffer.  The SmartPort write goes through SPDISPATCH, which saves
; and restores CSW/KSW/TXTPTR, so running it from inside the output hook is safe.
PRFLUSH:
        LDA PRLEN
        BEQ @done
        LDX #0                  ; stage the line into SP_PAYLOAD
@cp:    CPX PRLEN
        BEQ @go
        LDA PRLINEBUF,X
        STA SP_PAYLOAD,X
        INX
        BNE @cp
@go:
        LDY PRLEN               ; Y = byte count
```

```
        LDX SPPUNIT             ; X = printer unit
        JSR SP_WRITE
        LDA #0
        STA PRLEN
@done:
        RTS

;===============================================================================
; PUTS - print a NUL-terminated ASCII string at (A=lo, Y=hi).  The high bit is
; set on each byte for normal (non-inverse) COUT display; $00 terminates.
;===============================================================================
PUTS:
        STA STRP
        STY STRP+1
        LDY #0
@lp:    LDA (STRP),Y
        BEQ @done
        ORA #$80
        JSR COUT
        INY
        BNE @lp
@done:  RTS

PRINTBANNER:
        LDA #<MSG_BANNER
        LDY #>MSG_BANNER
        JMP PUTS

;===============================================================================
; PUTHEX - print A as two hex digits.
;===============================================================================
PUTHEX:
        TAX
        JMP PRTX

;===============================================================================
; CHKDEV - fail (SEC + A = NO DEVICE) if no FujiNet NETWORK device was found.
;===============================================================================
CHKDEV:
        LDA SPUNIT
        BNE @ok
        LDA #BE_NODEV
        SEC
        RTS
@ok:    CLC
        RTS

;===============================================================================
; POLLKEY - flow-control keyboard poll for NTYPE's output.
;   CTRL-S pauses: consume it, then wait here until the next key.  If that key
;          is CTRL-C, fall through to abort; any other key resumes.
;   CTRL-C aborts: consume it and return carry SET so the caller stops.
;   Any other key (or no key) is left queued; return carry CLEAR.
; Preserves X and Y (touches A only), so it can sit inside the byte loop.
;===============================================================================
POLLKEY:
        LDA KBD
        BPL @none               ; bit7 clear -> no key waiting
        CMP #CTRL_C
        BEQ @abort
        CMP #CTRL_S
        BNE @none               ; other key -> ignore, leave it queued
        BIT KBDSTRB             ; consume the CTRL-S
@wait:  LDA KBD                 ; paused: hold until the next key
        BPL @wait
        BIT KBDSTRB             ; consume the resume key
        CMP #CTRL_C             ; ...but CTRL-C while paused still aborts
        BEQ @abrt
@none:  CLC
        RTS
@abort: BIT KBDSTRB             ; consume the CTRL-C
@abrt:  SEC
        RTS
```

```
;================================================================================
; OPENCK - FN_OPEN on the already-selected channel, then verify the open
; actually succeeded.  The firmware's OPEN reply is ALWAYS "no error" (a
; failed protocol open only records its error in channel status - see
; iwmNetwork::open() in the firmware), so ask FN_STATUS and inspect the error
; byte: below 128, or 136 (EOF - an empty file is a fine open), is success.
; On failure: close the channel and return SEC with A = PATH NOT FOUND for
; err 170, I/O ERROR for anything else.
;
; Deliberately NOT used for HTTP(S) opens in H_NOPEN: an HTTP status call
; fires the pending transaction (the firmware performs the request on the
; first status/read), which would break the NHTTPMODE set-headers-then-fetch
; recipe.  HTTP open errors stay deferred to NSTATUS, as documented.
;================================================================================
OPENCK:
        JSR FN_OPEN
        JSR FN_STATUS           ; SP_PAYLOAD = [avail lo][avail hi][conn][err]
        LDA SP_PAYLOAD+3
        CMP #128
        BCC @ok                 ; informational -> open succeeded
        CMP #NERR_EOF
        BEQ @ok                 ; empty-but-open is not a failure
        STA OPENERR             ; failed: clean up, then map the code
        JSR FN_CLOSE
        LDA OPENERR
        CMP #NERR_FNF
        BEQ @fnf
        LDA #BE_IOERROR
        SEC
        RTS
@fnf:   LDA #BE_PATHNF
        SEC
        RTS
@ok:    CLC
        RTS

;================================================================================
; ISHTTP - carry set if the spec in URLBUF names an HTTP or HTTPS protocol:
; the four characters after the first ':' spell "HTTP" (case-insensitive).
;================================================================================
ISHTTP:
        LDX #0
@fc:    CPX URLLEN              ; find the first ':'
        BCS @no                 ; none -> not HTTP
        LDA URLBUF,X
        INX
        CMP #':'
        BNE @fc
        LDY #0
@cm:    LDA URLBUF,X
        AND #$DF                ; uppercase ASCII letters
        CMP HTTPSTR,Y
        BNE @no
        INX
        INY
        CPY #4
        BNE @cm
        SEC
        RTS
@no:    CLC
        RTS

;================================================================================
; CHKCHAN - validate the device number in X (1..15) and store it in CHAN.
; Returns CLC on success, or SEC + A = RANGE error.
;================================================================================
CHKCHAN:
        CPX #1
        BCC @bad
        CPX #16
        BCS @bad
        STX CHAN
```

```
        CLC
        RTS
@bad:
        LDA #BE_RANGE
        SEC
        RTS

;================================================================================
; GETURL - parse a string argument at TXTPTR (literal "..." or a string
; variable) into URLBUF, length in URLLEN.
;================================================================================
GETURL:
        JSR CHRGOT
        CMP #'"'
        BEQ @lit
        ; --- string variable ---
        JSR PTRGET              ; VARPTR -> descriptor [len][lo][hi]
        LDY #0
        LDA (VARPTR),Y
        STA URLLEN
        INY
        LDA (VARPTR),Y
        STA GUSRC
        INY
        LDA (VARPTR),Y
        STA GUSRC+1
        LDX #0
        LDY #0
@vcp:
        CPX URLLEN
        BEQ @done
        LDA (GUSRC),Y
        STA URLBUF,X
        INX
        INY
        BNE @vcp
@done:
        RTS
        ; --- quoted literal ---
        ; TXTPTR points AT the opening quote.  Copy the raw bytes after it up to
        ; the closing quote (or end-of-line), then advance TXTPTR past what we
        ; consumed so the caller's next CHKCOM sees the following delimiter.
        ;
        ; We must NOT use CHRGET here: it skips spaces and treats ':' as an
        ; end-of-statement marker, so it would truncate every "N:..." devicespec
        ; at the first colon and leave TXTPTR on the ':' (-> ?SYNTAX ERROR).
@lit:
        LDX #0
        LDY #1                  ; index 0 is the opening quote; body starts at 1
@lcp:
        LDA (TXTPTR),Y
        BEQ @lend               ; NUL -> end of line, no closing quote
        CMP #'"'
        BEQ @lclose
        STA URLBUF,X
        INX
        INY
        BNE @lcp
@lclose:
        INY                     ; step past the closing quote
@lend:
        STX URLLEN
        TYA                     ; advance TXTPTR by the Y bytes we scanned
        CLC
        ADC TXTPTR
        STA TXTPTR
        BCC @ldone
        INC TXTPTR+1
@ldone:
        RTS

;================================================================================
; WORDSAV / BYTESAV - store a value into the numeric variable named at TXTPTR.
```

```
;   WORDSAV: A = value high, Y = value low
;   BYTESAV: Y = value (0..255)
;================================================================================
BYTESAV:
        LDA #0
WORDSAV:
        JSR GIVAYF              ; (A=hi, Y=lo) -> FAC
        JSR PTRGET              ; parse variable name; A=addr lo, Y=addr hi
        TAX                     ; X = addr lo (Y already = addr hi)
        JSR MOVMF               ; pack FAC into the variable
        RTS

        .include "smartport.inc"

;================================================================================
; Keyword table.  Each entry: plain-ASCII name, $00, handler address (lo,hi).
; Terminated by a $00 first byte.  No entry may be a prefix of another (a
; trailing delimiter is required at match time, which also guards this).
;================================================================================
        .segment "RODATA"

CMDTABLE:
        .byte "NOPEN",0
        .addr H_NOPEN
        .byte "NCLOSE",0
        .addr H_NCLOSE
        .byte "NSTATUS",0
        .addr H_NSTATUS
        .byte "NREAD",0
        .addr H_NREAD
        .byte "NWRITE",0
        .addr H_NWRITE
        .byte "NJSONPARSE",0
        .addr H_NJSONPARSE
        .byte "NJSONQUERY",0
        .addr H_NJSONQUERY
        .byte "NCD",0
        .addr H_NCD
        .byte "NCAT",0
        .addr H_NCAT
        .byte "NCATALOG",0
        .addr H_NCATALOG
        .byte "NMKDIR",0
        .addr H_NMKDIR
        .byte "NRMDIR",0
        .addr H_NRMDIR
        .byte "NDEL",0
        .addr H_NDEL
        .byte "NLOAD",0
        .addr H_NLOAD
        .byte "NSAVE",0
        .addr H_NSAVE
        .byte "NACCEPT",0
        .addr H_NACCEPT
        .byte "NLOGIN",0
        .addr H_NLOGIN
        .byte "NHTTPMODE",0
        .addr H_NHTTPMODE
        .byte "NTRANS",0
        .addr H_NTRANS
        .byte "NTYPE",0
        .addr H_NTYPE
        .byte 0                 ; end of table

MSG_BANNER:
        .byte $8D               ; CR
        .byte "FUJINET BASIC EXTENSION INSTALLED",$8D
        .byte "20 COMMANDS - NOPEN..NTYPE",$8D
        .byte 0
MSG_NOROOM:
        .byte $8D,"FUJINET: NOT ENOUGH ROOM BELOW HIMEM",$8D,0
MSG_NETOK:
        .byte "FUJINET NETWORK DEVICE AT SP UNIT $",0
```

```
MSG_PROK:
        .byte "FUJINET PRINTER AT SP UNIT $",0
MSG_NOSP:
        .byte "SMARTPORT CARD NOT FOUND",$8D,0
MSG_NONET:
        .byte "FUJINET NETWORK DEVICE NOT FOUND",$8D,0

HTTPSTR:
        .byte "HTTP"            ; protocol sniff for ISHTTP

; --- the error trap's decoys (see CMDENTRY) ---
; FAKETGT is where HANDLERR points TXTPTR: the ASCII line number its GOTO
; parses.  FAKEPROG is where the swapped TXTTAB points: a one-line tokenized
; program, "0 CALL 32777", whose CALL lands in RECOVER via the entry table.
FAKETGT:
        .byte "0",0
FAKEPROG:
        .addr FPEND             ; line link
        .word 0                 ; line number 0
        .byte TOK_CALL,"32777",0
FPEND:  .word 0                 ; end-of-program marker

;============================================================================
; Resident variables + scratch buffers (uninitialized; protected by HIMEM).
;============================================================================
        .segment "BSS"

CMDSTART:   .res 1              ; index in $200 where the keyword starts
ARGSTART:   .res 1              ; index in $200 where the arguments start
HANDLERVEC: .res 2              ; real command handler addr (CMDENTRY jumps here)
SAVTXT:     .res 2              ; Applesoft TXTPTR saved across a command
SAVCURLIN:  .res 2              ; Applesoft CURLIN saved across a command
SAVONERR:   .res 6              ; ERRFLG + ONRTXT + ONRLIN + TRCFLG across a command
SAVTTAB:    .res 2              ; the REAL TXTTAB while the error trap is armed
TRAPSP:     .res 1              ; stack level RECOVER unwinds to

; --- transport / command state ---
SPDISP:     .res 2              ; SmartPort dispatch address (JMP (SPDISP))
SPUNIT:     .res 1              ; SmartPort unit # of the "NETWORK" device (0=none)
SPPUNIT:    .res 1              ; SmartPort unit # of the "PRINTER" device (0=none)
SAVVEC:     .res 2              ; output device saved by PRON, restored by PROFF
PRLEN:      .res 1              ; printer line-buffer fill level
PRCH:       .res 1              ; printer hook: saved output character
FDNAMEBUF:  .res 7              ; FINDDEV: 7-char target device name
PRLINEBUF:  .res 88             ; printer line buffer (flushed at 80 / on CR)
DCOUNT:     .res 1              ; SmartPort device count
CURDEV:     .res 1              ; device being probed by FINDNET
SP_COUNT:   .res 2              ; bytes moved by the last SmartPort read/write
SAVHK:      .res 6              ; saved CSW/KSW ($36-$39) + TXTPTR ($B8/$B9)
FNCTMP:     .res 1              ; scratch: saved control code
NCTLCMD:    .res 1              ; NCTL: control code held across devicespec parse
OPENERR:    .res 1              ; OPENCK: channel error held across the cleanup close
JQRETRY:    .res 1              ; NJSONQUERY: status-poll retry counter
SVLEN:      .res 2              ; NSAVE: bytes of program left to write
NEXTPTR:    .res 2              ; RELINK: next-line address scratch
CHAN:       .res 1              ; current channel (1..15)
MODE:       .res 1              ; open mode
TRANS:      .res 1              ; open translation
CURTRANS:   .res 1              ; NTRANS: standing translation for NTYPE (chan 1)
BUFLEN:     .res 1              ; requested read/write length
URLLEN:     .res 1              ; length of URLBUF contents
RDVAR:      .res 2              ; saved buf$ descriptor address across a read
CMD_LIST:   .res 16             ; SmartPort parameter list

; --- big buffers ---
ARGBUF:     .res 256            ; clean (hi-bit-stripped, NUL-term) argument copy
                                ; (also the install-time zero-page snapshot)
URLBUF:     .res 256            ; parsed devicespec / string argument
SP_PAYLOAD: .res 320            ; SmartPort control-list / read-write data buffer

RES_END:                        ; end of the resident image (for the room check)
```

```
;*******************************************************************************
; smartport.inc - SmartPort transport for the FujiNet network device
;
; The current FujiNet firmware exposes ONE SmartPort device named "NETWORK"
; (lib/device/iwm/network.cpp).  Channels (N1, N2, ...) are sub-selected by
; NETCMD_SET_CHANNEL (0xFA) rather than by separate devices.  We therefore:
;   1. find the SmartPort card slot and its dispatch entry (FINDSP)
;   2. find the "NETWORK" device's SmartPort unit number (FINDNET)
;   3. before each channel op, set the active channel (FN_SETCH)
;
; SmartPort call convention (JSR the dispatcher, inline cmd + param list):
;       JSR CALL_DISPATCHER
;       .byte <cmd>
;       .word <param list>
; On return: A = SmartPort error, carry set on error, X/Y = byte count.
;
; Control lists are length-prefixed: SP_PAYLOAD[0..1] = data byte count,
; SP_PAYLOAD[2..] = the bytes the firmware sees as data_buffer[0..].
; Status/read replies are written straight into SP_PAYLOAD[0..].
;*******************************************************************************

;-------------------------------------------------------------------------------
; CALL_DISPATCHER - transfer to the SmartPort firmware entry.  The inline
; cmd/param-list bytes that follow each JSR are consumed by the firmware.
;-------------------------------------------------------------------------------
CALL_DISPATCHER:
        JMP (SPDISP)

;-------------------------------------------------------------------------------
; FINDSP - locate the SmartPort card (slots 7..1) and compute its dispatch
; address into SPDISP.  Returns carry clear on success, carry set if none.
;-------------------------------------------------------------------------------
FINDSP:
        LDA #$C7                ; start at slot 7 ($C700)
        STA SLOTPTR+1
        LDA #$00
        STA SLOTPTR
@scan:
        LDY #$01                ; signature bytes live at offsets 1,3,5,7
        LDX #$00
@match:
        LDA (SLOTPTR),Y
        CMP SP_SIG,X
        BNE @nextslot
        INY
        INY
        INX
        CPX #$04
        BNE @match
        ; found: SLOTPTR = $Cn00.  dispatch = $Cn00 + [$CnFF] + 3
        LDY #$FF
        LDA (SLOTPTR),Y
        CLC
        ADC SLOTPTR
        STA SPDISP
        LDA SLOTPTR+1
        ADC #$00
        STA SPDISP+1
        LDA SPDISP
        CLC
        ADC #$03
        STA SPDISP
        LDA SPDISP+1
        ADC #$00
        STA SPDISP+1
        CLC
        RTS
@nextslot:
```

```
        DEC SLOTPTR+1             ; $C700 -> $C600 -> ...
        LDA SLOTPTR+1
        CMP #$C0                  ; below slot 1?
        BNE @scan
        SEC                       ; not found
        RTS

;-------------------------------------------------------------------------
; SPDISPATCH - issue the SmartPort call whose command number is in A, with the
; parameter list already built in CMD_LIST.
;
; A direct SmartPort call can leave CSW/KSW ($36-$39) clobbered (the card uses
; page zero as scratch, and may reset the output link to the standard COUT).
; That silently unhooks BASIC.SYSTEM's command wedge, after which CATALOG /
; OPEN / our N-commands all fall through to Applesoft as ?SYNTAX ERROR.  So we
; save $36-$39 before the call and restore them after.  (The original ext.
; escaped this because it ran under plain Applesoft, where a standard COUT link
; is fine.)
;
; Returns A = error, carry = error, X/Y = byte count (also stored in SP_COUNT).
;-------------------------------------------------------------------------
SPDISPATCH:
        STA SPCMDN                ; self-modify the inline command byte
        LDX #3                    ; save CSW/KSW contents ($36-$39, contiguous)
@sv:    LDA CSW,X
        STA SAVHK,X
        DEX
        BPL @sv
        LDA TXTPTR                ; save TXTPTR ($B8/$B9); parse resumes after us
        STA SAVHK+4
        LDA TXTPTR+1
        STA SAVHK+5
        JSR CALL_DISPATCHER
SPCMDN: .byte 0                   ; <- patched command number
        .addr CMD_LIST
        ; firmware returns here: A=error, carry, X=count lo, Y=count hi
        STX SP_COUNT
        STY SP_COUNT+1
        PHP                       ; preserve error/carry across the restore
        PHA
        LDX #3                    ; restore CSW/KSW ($36-$39)
@rs:    LDA SAVHK,X
        STA CSW,X
        DEX
        BPL @rs
        LDA SAVHK+4               ; restore TXTPTR
        STA TXTPTR
        LDA SAVHK+5
        STA TXTPTR+1
        PLA
        PLP
        RTS

;-------------------------------------------------------------------------
; SmartPort primitives.  Each sets up CMD_LIST then tail-calls SPDISPATCH.
;   SP_STATUS  X=unit  Y=status code   -> reply in SP_PAYLOAD, count in X/Y
;   SP_CONTROL X=unit  Y=control code  -> control list in SP_PAYLOAD
;   SP_READ    X=unit  Y=byte count    -> data in SP_PAYLOAD, count in SP_COUNT
;   SP_WRITE   X=unit  Y=byte count    -> data taken from SP_PAYLOAD
;-------------------------------------------------------------------------
SP_STATUS:
        LDA #3
        STA CMD_LIST              ; param count
        STX CMD_LIST+1            ; unit
        LDA #<SP_PAYLOAD
        STA CMD_LIST+2            ; status list pointer
        LDA #>SP_PAYLOAD
        STA CMD_LIST+3
        STY CMD_LIST+4            ; status code
        LDA #SP_CMD_STATUS
        JMP SPDISPATCH

SP_CONTROL:
```

```
        LDA #3
        STA CMD_LIST
        STX CMD_LIST+1          ; unit
        LDA #<SP_PAYLOAD
        STA CMD_LIST+2          ; control list pointer
        LDA #>SP_PAYLOAD
        STA CMD_LIST+3
        STY CMD_LIST+4          ; control code
        LDA #SP_CMD_CONTROL
        JMP SPDISPATCH

SP_READ:
        LDA #4
        STA CMD_LIST
        STX CMD_LIST+1          ; unit
        LDA #<SP_PAYLOAD
        STA CMD_LIST+2          ; data buffer pointer
        LDA #>SP_PAYLOAD
        STA CMD_LIST+3
        STY CMD_LIST+4          ; byte count lo
        LDA #0
        STA CMD_LIST+5          ; byte count hi (reads capped at 255)
        STA CMD_LIST+6          ; address (unused for network)
        STA CMD_LIST+7
        STA CMD_LIST+8
        LDA #SP_CMD_READ
        JMP SPDISPATCH

SP_WRITE:
        LDA #4
        STA CMD_LIST
        STX CMD_LIST+1          ; unit
        LDA #<SP_PAYLOAD
        STA CMD_LIST+2          ; data buffer pointer
        LDA #>SP_PAYLOAD
        STA CMD_LIST+3
        STY CMD_LIST+4          ; byte count lo
        LDA #0
        STA CMD_LIST+5          ; byte count hi
        STA CMD_LIST+6
        STA CMD_LIST+7
        STA CMD_LIST+8
        LDA #SP_CMD_WRITE
        JMP SPDISPATCH

;-------------------------------------------------------------------------------
; FINDDEV - scan the SmartPort devices for one whose 7-char DIB name matches the
; 7 bytes at FDNAMEBUF.  Compared with absolute addressing (no zero-page pointer
; at risk across the SP_STATUS calls).  Returns the matching unit number in A,
; or 0 if not found.
;-------------------------------------------------------------------------------
FINDDEV:
        LDX #0                  ; unit 0, status code 0 -> device count
        LDY #SP_STATUS_CODE
        JSR SP_STATUS
        LDA SP_PAYLOAD          ; first status byte = number of devices
        STA DCOUNT
        LDX #1                  ; devices are numbered from 1
@loop:
        STX CURDEV
        LDY #SP_STATUS_DIB      ; device information block
        JSR SP_STATUS
        LDA SP_PAYLOAD+4        ; DIB id-string length
        CMP #7                  ; both "NETWORK" and "PRINTER" are 7 chars
        BNE @next
        LDX #0
@cmp:
        LDA SP_PAYLOAD+5,X
        CMP FDNAMEBUF,X
        BNE @next
        INX
        CPX #7
        BNE @cmp
```

```
        LDA CURDEV              ; matched
        RTS
@next:
        LDX CURDEV
        INX
        CPX DCOUNT
        BCC @loop               ; more devices below the count
        BEQ @loop               ; also test the last device (unit == count)
        LDA #0                  ; exhausted, not found
        RTS

;--------------------------------------------------------------------------
; FINDNET / FINDPRINTER - copy the target name into FDNAMEBUF, locate the unit
; via FINDDEV, and store it in SPUNIT / SPPUNIT (0 = not found).
;--------------------------------------------------------------------------
FINDNET:
        LDX #6
@cp:    LDA NETNAME,X
        STA FDNAMEBUF,X
        DEX
        BPL @cp
        JSR FINDDEV
        STA SPUNIT
        RTS

FINDPRINTER:
        LDX #6
@cp:    LDA PRNAME,X
        STA FDNAMEBUF,X
        DEX
        BPL @cp
        JSR FINDDEV
        STA SPPUNIT
        RTS

;==========================================================================
; FujiNet channel operations, layered on the primitives above.
;
; SP_PAYLOAD is shared, and FN_SETCH runs a full SET_CHANNEL round-trip using
; it.  So the rule is: a command handler calls FN_SETCH ONCE (right after it
; parses the channel), and the operations below assume the active channel is
; already selected and never call FN_SETCH themselves.  That way an operation
; is free to stage data into SP_PAYLOAD without a later SET_CHANNEL clobbering
; it.
;==========================================================================

;--------------------------------------------------------------------------
; FN_SETCH - make channel A the active network unit (NETCMD_SET_CHANNEL).
;--------------------------------------------------------------------------
FN_SETCH:
        STA SP_PAYLOAD+2        ; data_buffer[0] = channel
        LDA #1
        STA SP_PAYLOAD          ; count = 1 byte
        LDA #0
        STA SP_PAYLOAD+1
        LDX SPUNIT
        LDY #NETCMD_SET_CHANNEL
        JMP SP_CONTROL

;--------------------------------------------------------------------------
; FN_OPEN - open the active channel to URLBUF (URLLEN bytes) with MODE / TRANS.
;--------------------------------------------------------------------------
FN_OPEN:
        LDA MODE
        STA SP_PAYLOAD+2        ; data_buffer[0] = mode
        LDA TRANS
        STA SP_PAYLOAD+3        ; data_buffer[1] = trans
        LDX #0                  ; copy URL -> data_buffer[2..]
@cp:
        CPX URLLEN
        BEQ @cpdone
        LDA URLBUF,X
        STA SP_PAYLOAD+4,X
```

```
        INX
        BNE @cp
@cpdone:
        LDA #0
        STA SP_PAYLOAD+4,X      ; NUL terminate
        ; count = mode + trans + url + NUL = URLLEN + 3
        LDA URLLEN
        CLC
        ADC #3
        STA SP_PAYLOAD
        LDA #0
        ADC #0
        STA SP_PAYLOAD+1
        LDX SPUNIT
        LDY #NETCMD_OPEN
        JMP SP_CONTROL

;---------------------------------------------------------------------------
; FN_CLOSE - close the active channel.
;---------------------------------------------------------------------------
FN_CLOSE:
        LDA #0
        STA SP_PAYLOAD          ; empty control list
        STA SP_PAYLOAD+1
        LDX SPUNIT
        LDY #NETCMD_CLOSE
        JMP SP_CONTROL

;---------------------------------------------------------------------------
; FN_STATUS - query the active channel; on return
; SP_PAYLOAD = [avail_lo][avail_hi][conn][err]
;---------------------------------------------------------------------------
FN_STATUS:
        LDX SPUNIT
        LDY #NETCMD_STATUS
        JMP SP_STATUS

;---------------------------------------------------------------------------
; FN_READ - read A bytes from the active channel into SP_PAYLOAD;
; SP_COUNT = actual bytes read.
;---------------------------------------------------------------------------
FN_READ:
        TAY
        LDX SPUNIT
        JMP SP_READ

;---------------------------------------------------------------------------
; FN_WRITE - write A bytes from SP_PAYLOAD[0..] to the active channel.  The
; caller must have staged the data in SP_PAYLOAD already (after FN_SETCH).
;---------------------------------------------------------------------------
FN_WRITE:
        TAY
        LDX SPUNIT
        JMP SP_WRITE

;---------------------------------------------------------------------------
; FN_CONTROL_DS - control command whose data is the devicespec in URLBUF/URLLEN
; (used by NCD/NMKDIR/NRMDIR/NDEL).  Y = control code.  The caller has already
; selected the channel.
;---------------------------------------------------------------------------
FN_CONTROL_DS:
        STY FNCTMP              ; save control code
        LDX #0
@cp:
        CPX URLLEN
        BEQ @done
        LDA URLBUF,X
        STA SP_PAYLOAD+2,X
        INX
        BNE @cp
@done:
        LDA #0
        STA SP_PAYLOAD+2,X      ; NUL terminate
```

```
        LDA URLLEN              ; count = URLLEN + 1 (NUL)
        CLC
        ADC #1
        STA SP_PAYLOAD
        LDA #0
        ADC #0
        STA SP_PAYLOAD+1
        LDX SPUNIT
        LDY FNCTMP
        JMP SP_CONTROL

NETNAME: .byte "NETWORK"
PRNAME:  .byte "PRINTER"
SP_SIG:  .byte $20,$00,$03,$00
```